HARDWARE AND SYSTEMS

Hardware and low-level systems from first principles

UNIT 3

Control, Radio and Boards

Motors, radio links, microwave radar, printed circuit boards and a drone that flies on them

3.1 Motors, estimation and feedback control

3.2 Radio links

3.3 Microwave synthesis and FMCW radar

3.4 Board design and bring-up

3.5 The drone: integration, test and flight

Second edition, October 2026

Contents

About this unit 2
- Before you start 2
- Kit for this unit 2

3.1 Motors, estimation and feedback control 3
- Core ideas 3
- Reading 7
- Labs 7
- Problem set 11
- Deliverables and stretch 11

3.2 Radio links 12
- Core ideas 12
- Reading 15
- Labs 15
- Problem set 18
- Deliverables and stretch 19

3.3 Microwave synthesis and FMCW radar 20
- Core ideas 20
- Reading 23
- Labs 23
- Problem set 26
- Deliverables and stretch 26

3.4 Board design and bring-up 28
- Core ideas 28
- Reading 31
- Labs 31
- The design review checklist 35
- Problem set 36
- Deliverables and stretch 36

3.5 The drone: integration, test and flight 38
- Core ideas 38
- Reading 41
- Labs 41
- Problem set 45
- Deliverables and stretch 46

Sign-off 47

UNIT 3

About this unit

In Unit 3 the course's hardware work moves to systems whose behaviour you can see and hear. You write field-oriented motor control on a C2000, characterise an IMU and close attitude loops on a rig. You build a packet radio below TI's stacks and prove its sensitivity, program a microwave synthesiser, and configure TI's 60 GHz radar as an altimeter. You design, fabricate and bring up four boards, two for the drone and two that become Module Zero's first node. Finally you integrate everything into a quadcopter that hovers, holds altitude and reports telemetry over your own link.

Chapter	You build	Weeks
3.1	A thrust stand, FOC from scratch, an IMU noise model, an attitude estimator, PID on a rig	4
3.2	A packet radio, a measured sensitivity waterfall, a telemetry protocol, an SDR demodulator	3
3.3	A synthesiser you program, a measured chirp, an FMCW simulator, a radar altimeter	3
3.4	Boards P, F, A and B, specified, reviewed, fabricated and brought up	6 + fabrication
3.5	DShot, CRSF and the flight software; assembly, tuning, hover and altitude hold	5

Before you start

Set up the Windows PC before Chapter 3.2 and read the Handbook's safety rules on lithium packs and propellers before Chapter 3.1. Board A's specification can start as soon as Chapter 2.6 is done, to keep fabrication off the critical path.

Kit for this unit

The Handbook's kit part lists every item for this unit with its part number, seller and price. The totals, for items this unit needs first:

	Required	Optional
UK sellers, VAT included (ex-VAT prices × 1.2)	£1,648	£154
Overseas sellers, before import VAT and delivery	US$199	US$125
Items with no checked price	1	none

Not priced: Compact USB power bank, about 5,000 mAh, no longer than 105 mm, bought locally.

CHAPTER 3.1

Motors, estimation and feedback control

TIME	4 weeks
YOU BUILD	A calibrated thrust stand and a measured model of the drone's motor and propeller; field-oriented control written from scratch on a C2000 microcontroller, compared against a commercial ESC; an IMU driver with a measured noise model; an attitude estimator running on the flight-controller LaunchPad; a calibrated magnetometer; a PID controller that holds the drone frame level on a one-axis rig, designed in simulation first.
NEEDS	The Chapter 1.2 load-cell rig with the ADS1220, a second INA228 with the Bourns 1 mΩ shunt, the LAUNCHXL-F280025C, BOOSTXL-DRV8323RS and LVBLDCMTR, two SunnySky motors with props, two CloudPhoenix ESCs, the 3S pack and charger, an MSP-EXP432E401Y, the LSM6DSOX and LIS3MDL breakouts, the Tarot frame, the oscilloscope, the bench supply, safety glasses.

WHAT THIS BUYS OMH

A server module and a drone are both machines that hold a variable at a setpoint against disturbances: inlet temperature through fan speed, attitude through motor thrust. Fan control, power capping, PCIe link training and the drone's flight controller all rest on the same three skills: measuring the thing you control, estimating what you cannot measure, and closing a loop with known stability margins. This chapter teaches them on hardware where a bad loop is visible and audible. It also builds the drone's propulsion model, its attitude estimator and its first controller, which Chapter 3.5 assembles into a drone that flies.

SKIP TEST

1. A propeller gives 300 g of thrust at 5,000 rpm. Assuming thrust proportional to the square of speed, what speed gives 600 g, and roughly how does the electrical power scale?
2. Write the Clarke and Park transforms, and explain why a current controller in the rotating frame needs only DC setpoints in steady state.
3. A gyroscope has a constant bias of 0.5 °/s. What is the attitude error after 60 s of pure integration, and how does a complementary filter bound it?
4. Derive the discrete PID update with a filtered derivative on the measurement and integrator clamping. Why differentiate the measurement rather than the error?
5. A loop has 45° of phase margin at a crossover of 20 Hz. How much pure delay can you add before it becomes unstable?

If all five are easy, do Labs 3.1.1, 3.1.5 and 3.1.7 only.

Core ideas

Propulsion. A brushless motor with a fixed-pitch propeller produces thrust $T = k_T\omega^2$ and drag torque $Q = k_Q\omega^2$, so mechanical power is $Q\omega = k_Q\omega^3$. Doubling thrust needs $\sqrt{2}$ times the speed and about $2^{1.5} \approx 2.8$ times the power, which is why a heavy drone's flight time falls quickly. The motor's KV constant gives no-load speed per volt; the SunnySky X2212-13 is 980 KV, so on a 3S pack it spins at most about 12,000 rpm before load. The figures that matter for a design are measured: thrust per watt at hover, the current at full throttle, and the time constant from a throttle step to a thrust step. The last one limits how fast any attitude controller can be.

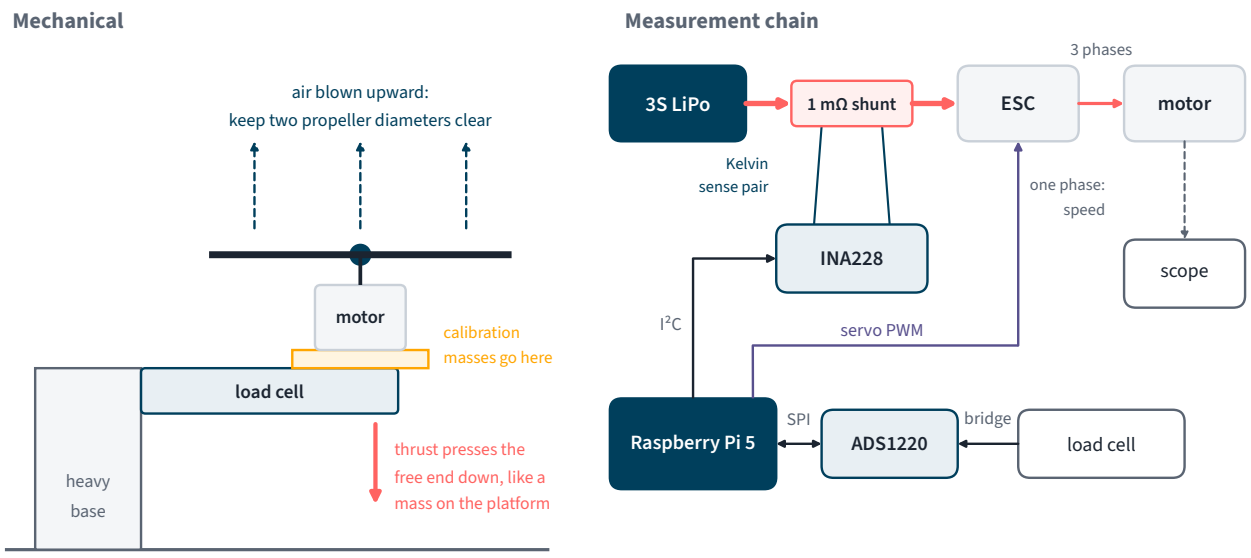


Figure 3.1.1. The thrust stand. The propeller blows upward, so thrust presses down on the load cell exactly as a calibration mass does. The INA228 measures the ESC's input power through the 1 mΩ shunt with a Kelvin connection.

Brushless DC motors. A brushless motor has three phase windings on the stator and permanent magnets on the rotor (outrunners such as the X2212 put the magnets on a rotating bell). Counting magnets gives the pole count: a typical 2212 outrunner has 14 magnets, 7 pole pairs, so one mechanical revolution is seven electrical cycles. Each phase is a resistance R, an inductance L and a back-EMF proportional to speed. A six-step (trapezoidal) drive energises two phases at a time and times commutation from the back-EMF zero crossing on the floating phase; hobby ESCs such as the CloudPhoenix do this, or a refined version of it. Field-oriented control (FOC) drives all three phases with sinusoidal currents whose vector is held at 90 electrical degrees to the rotor's magnetic axis, which gives the most torque per amp and the quietest operation.

Field-oriented control. Measure two phase currents (the third is their negative sum), transform them to a stationary two-axis frame with the Clarke transform, then rotate them into the rotor frame with the Park transform:

$$i_\alpha = i_a, \quad i_\beta = \frac{i_a + 2i_b}{\sqrt{3}}, \quad i_d = i_\alpha \cos\theta + i_\beta \sin\theta, \quad i_q = -i_\alpha \sin\theta + i_\beta \cos\theta$$

In the rotor frame the currents are DC in steady state. Torque is $\tau = \frac{3}{2} p \lambda_m i_q$ for p pole pairs and magnet flux linkage λ_m, so you command i_q for torque and hold $i_d = 0$. Two PI controllers produce v_d and v_q; the inverse Park transform and space-vector modulation turn them into three PWM duty cycles. Tune each current loop by pole-zero cancellation: $K_p = L\omega_c$ and $K_i = R\omega_c$ in continuous time for a crossover ω_c, kept below about a tenth of the PWM frequency. A speed loop, slower again, sets i_q.

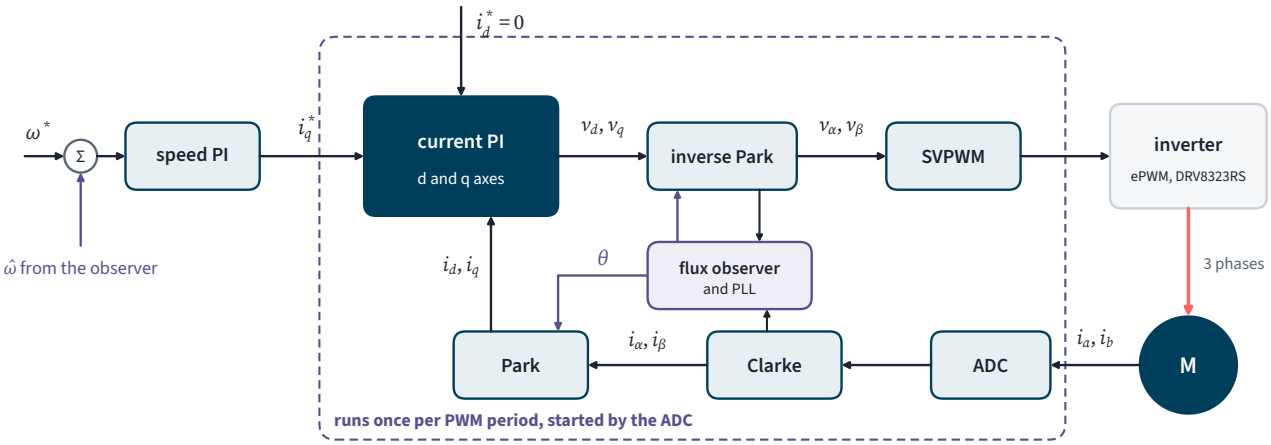


Figure 3.1.2. Field-oriented control as Lab 3.1.2 builds it. Everything inside the dashed box runs once per PWM period, triggered by the ADC conversion that the PWM itself starts.

Estimating the rotor angle. A sensorless drive estimates θ from what it measures. Integrate the stator voltage minus the resistive drop to get flux, subtract Li, and the remainder is the magnet's flux vector, whose angle is the rotor angle: $\psi_{\alpha\beta} = \int \left(v_{\alpha\beta} - R i_{\alpha\beta}\right) \mathrm{d}t - L i_{\alpha\beta}$. A phase-locked loop on that angle gives a clean speed. The method fails at standstill, where there is no back-EMF, so the motor starts in open loop (a rotating current vector the rotor follows) and hands over to the observer above a few hundred rpm. TI's InstaSPIN FAST estimator and its enhanced sliding-mode observer (eSMO), both in the C2000 Motor Control SDK, are industrial versions of the same idea; you will compare yours against them.

Timing. Centre-aligned PWM puts all three low-side switches on at the counter's zero, so the low-side shunts see the phase currents then. The ePWM module triggers the ADC at that instant, the ADC interrupt runs the whole control step, and the new duty cycles load at the next period. At a 20 kHz PWM rate the budget is 50 µs including the trigonometry, which is why the C2000 has a trigonometric math unit. Dead time between the high and low switches prevents shoot-through and distorts the output voltage slightly at low current, which a careful controller compensates.

Inertial sensors. A MEMS gyroscope measures angular rate with a vibrating proof mass and the Coriolis force; a MEMS accelerometer measures specific force, which is gravity plus any acceleration. ST's LSM6DSOX packs both, with programmable ranges, output rates up to 6.66 kHz, on-chip anti-alias and digital filters, and a FIFO. Their errors are those of Chapter 1.3 with time added: bias, bias drift with temperature, scale factor, cross-axis sensitivity, and noise. Gyro noise integrates into an attitude random walk; gyro bias integrates into an attitude error that grows linearly. An Allan deviation plot separates these terms from a long static log: the slope of $-\frac{1}{2}$ at short averaging times is angle random walk, and the flat minimum is bias instability.

Vibration and aliasing. Motors and propellers shake the frame at the rotation frequency and its harmonics, from about 80 Hz at hover to several hundred hertz. If the IMU's output rate and filters let that through, it aliases into low frequencies that look like real motion, as in Chapter 1.2. Soft mounting, the sensor's own low-pass filters, a high output rate with digital filtering, and notch filters that track motor speed are the standard defences.

Attitude estimation. The gyroscope is accurate over short times but drifts. The accelerometer's estimate of the gravity direction is noisy and corrupted by acceleration but does not drift. A complementary filter blends them with crossover frequency ω_c: high-pass the integrated gyro, low-pass the accelerometer angle. In discrete form, $\hat{\theta}_k = \alpha\left(\hat{\theta}_{k-1} + \omega_k \Delta t\right) + (1-\alpha)\theta_{a,k}$ with $\alpha = \tau/(\tau + \Delta t)$. In three dimensions you integrate a quaternion and correct it towards the accelerometer's gravity vector (and the magnetometer's north) with a proportional-integral term; this is Mahony's filter, and the integral term estimates the gyro bias. A Kalman filter does the same blending with a model of each noise source and is optimal when that model is right.

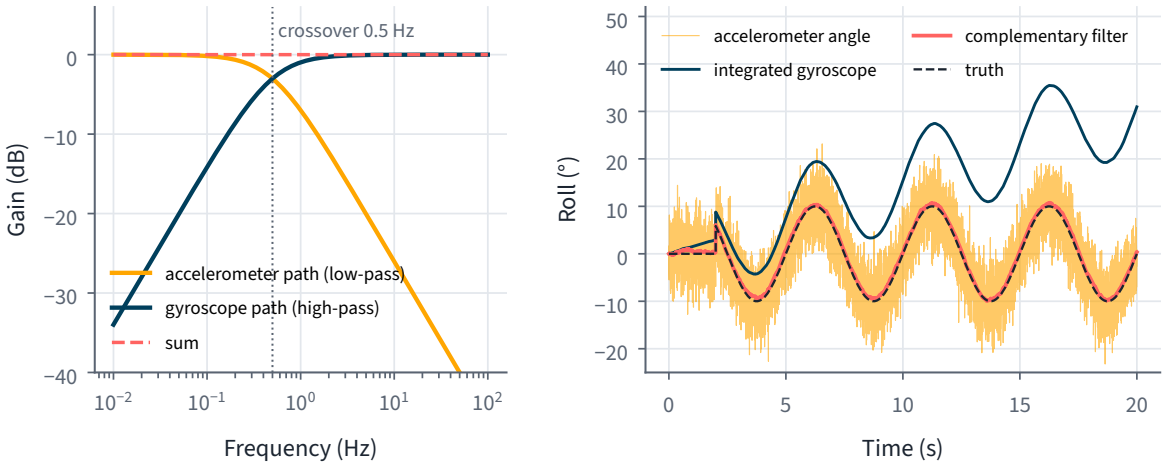


Figure 3.1.3. A complementary filter's two paths. The gyroscope is trusted above the crossover frequency and the accelerometer below it; the sum passes every frequency with unit gain.

Magnetometers. A compass needs a magnetoresistive sensor, as Chapter 1.3 showed. The LIS3MDL measures Earth's field easily, but on a drone it also measures the frame's steel screws (hard iron, a constant offset), nearby ferrous material that bends the field (soft iron, an ellipsoidal distortion), and the field of the battery current, which changes with throttle. Calibration fits an ellipsoid to readings taken in many orientations; the current term is measured and either compensated or avoided by mounting the sensor on a mast.

Feedback control. A plant with transfer function $G(s)$ and a controller $C(s)$ in unity feedback have loop gain $L = CG$. The crossover frequency, where $|L| = 1$, sets the closed-loop bandwidth; the phase margin there sets how much the response overshoots and how much delay the loop tolerates, $\Delta t_{\max} = \mathrm{PM}\,/\omega_c$ with the margin in radians. Every filter, every sample period and every millisecond of motor lag takes phase. PID is a pragmatic controller: proportional for stiffness, integral for zero steady-state error, derivative for damping. In a digital implementation, differentiate the measurement (so a setpoint step does not kick), low-pass the derivative, clamp or back-calculate the integrator so it does not wind up when the motors saturate, and add feedforward where you know the plant.

Cascaded loops. Drones use a fast inner loop on angular rate (gyro only, 1 to 8 kHz) and a slower outer loop on angle (estimator output, a few hundred hertz) whose output is the inner loop's setpoint. Each outer loop should be at least three to five times slower than the loop inside it. A mixer converts the roll, pitch, yaw and collective thrust commands into four motor commands, and must decide what to give up when a motor saturates.

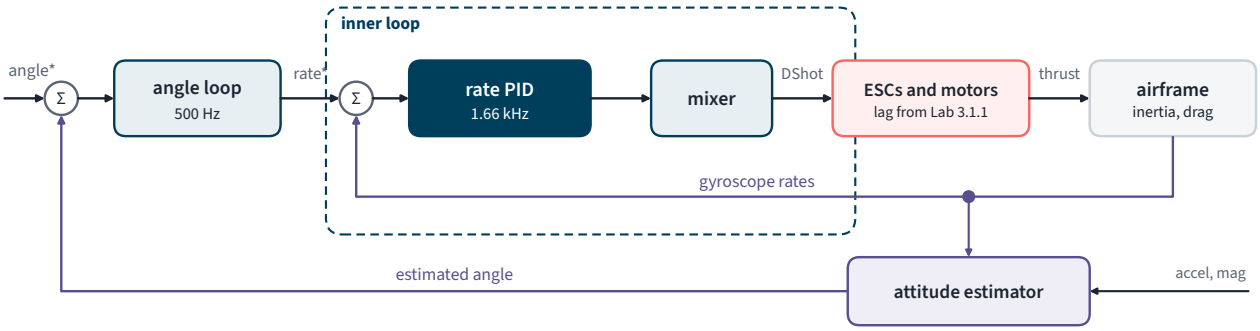


Figure 3.1.4. The cascade you build on the rig in Lab 3.1.7 and fly in Chapter 3.5. The motor's own lag, measured in Lab 3.1.1, is part of the plant the rate loop must control.

Design in simulation, verify on the rig. A one-axis model is two lines: $J\ddot{\theta} = l(T_1 - T_2) - b\dot{\theta}$ with each motor's thrust lagging its command by the measured time constant. With J from the frame's geometry and a swing test, l the arm length, and the motor model from the thrust stand, you can choose gains, predict the step response, and see what a 4 ms delay does before you power anything.

Reading

- Hanselman, *Brushless Motors: Magnetics, Design and Control* (2012), chapters 1 to 4, or the first half of Krishnan, *Permanent Magnet Synchronous and Brushless DC Motor Drives*.
- TI SPRUJ26 (the C2000 Motor Control SDK universal lab guide), the sections on the build levels, current-loop tuning and the eSMO observer; TI's DRV8323RS datasheet sections on the gate driver, current-sense amplifiers and SPI registers; the BOOSTXL-DRV8323Rx user's guide (SLVUB01C) and schematic.
- TI TIDUCF1, *High-Speed Sensorless-FOC Reference Design for Drone ESCs*, for what a production drone ESC needs (45 kHz PWM, electrical frequencies above 1 kHz).
- Åström and Murray, *Feedback Systems* (free online), chapters 1, 8 to 11 and 13; or Franklin, Powell and Emami-Naeini, *Feedback Control of Dynamic Systems*, chapters 3, 4, 6 and 8.
- Mahony, Hamel and Pflimlin, "Nonlinear Complementary Filters on the Special Orthogonal Group" (IEEE TAC, 2008), sections I to IV; Madgwick's 2010 report on the gradient-descent orientation filter.
- IEEE Std 952 Annex C, or El-Sheimy et al., "Analysis and Modeling of Inertial Sensors Using Allan Variance" (IEEE TIM, 2008).
- ST's LSM6DSOX datasheet and application note AN5272; ST's LIS3MDL datasheet.
- Mellinger and Kumar, "Minimum Snap Trajectory Generation and Control for Quadrotors" (ICRA 2011), section II, or chapters 2 and 3 of Beard and McLain, *Small Unmanned Aircraft*, for the rigid-body model Chapter 3.5 needs.

Labs

SAFETY

Spinning propellers cut. Wear safety glasses for every powered motor test. Fit propellers only when the motor is clamped to the stand or the rig, stand out of the propeller's plane, and arm only after you have checked that your kill switch disarms. Remove propellers before you flash firmware, change wiring or debug. The LiPo rules from Chapter 1.4 apply to every lab that uses the pack.

LAB 3.1.1 The thrust stand 8 h

Goal. measure the drone's propulsion so that every later design uses measured numbers.

Kit. Load-cell rig and ADS1220 from Lab 1.2.4, Raspberry Pi 5, SunnySky motor and 10 × 4.5 prop, one CloudPhoenix ESC, the 3S pack, the second INA228 with its on-board shunt removed and the Bourns 1 mΩ shunt wired in its place, oscilloscope, calibration masses, safety glasses.

1. Rebuild the load-cell rig as a stand: the bar's fixed end bolted to a heavy base, a plywood platform on the free end, the motor on the platform with the propeller blowing air upward, and at least two propeller diameters of clear air above it. Calibrate with masses placed on the platform from 0 to 1 kg, and check linearity.
2. Desolder the 15 mΩ shunt from the second INA228 board, wire its IN+ and IN− to the Bourns shunt's sense screws with a twisted pair, and recompute your calibration from Lab 1.3.3 for 1 mΩ and 20 A full scale.
3. Drive the ESC from one of the Pi's hardware PWM channels with standard servo pulses (1,000 to 2,000 µs at 50 Hz) and calibrate the ESC's throttle range. Probe one motor phase with the scope and compute speed from the electrical frequency and the pole-pair count you counted on the bell.
4. Sweep throttle in 10 % steps to 80 %, holding each for 3 s and logging thrust, voltage, current and speed. Fit k_T from thrust against speed squared, and plot grams per watt against thrust. Mark the hover point for a 1.2 kg drone.
5. Step throttle from 30 % to 50 % and back while sampling the ADS1220 at 2,000 samples per second in turbo mode. Fit a first-order time constant to the thrust response, up and down.

DONE WHEN

- ☐ The calibration is linear to 0.5 % of full scale, and the sweep table and fitted k_T are in the notebook.
- ☐ Grams per watt at hover thrust is stated with its uncertainty and compared with the maker's table.
- ☐ The thrust time constant is measured for steps up and down.

EVIDENCE FOR THE NOTEBOOK

- Stand photographs; calibration data; sweep CSV and plots; the step-response fits.

LAB 3.1.2 Field-oriented control from scratch

3 weekends

Goal. drive a motor with control code you write, running the control step 20,000 times a second.

Kit. LAUNCHXL-F280025C, BOOSTXL-DRV8323RS, LVBLDCMTR, bench supply at 24 V limited to 2 A, oscilloscope, Code Composer Studio with C2000Ware.

1. Read the BoosterPack schematic and record the shunt resistance, the current-sense amplifier's gain options and the phase-voltage divider ratios. Seat the BoosterPack on the LaunchPad site TI's Motor Control SDK documents for this pair. Configure the DRV8323RS over SPI: gate drive current, dead time, overcurrent response, and a sense-amplifier gain that maps 5 A to most of the ADC's range.
2. Use C2000Ware's driverlib for peripheral set-up only; write the control code yourself. Configure three ePWM modules for centre-aligned complementary PWM at 20 kHz with dead band, and trigger ADC conversions of the three currents and the bus voltage at the counter's zero. Toggle a GPIO in the ADC interrupt and measure the timing on the scope.
3. Calibrate the current offsets with the gate driver disabled. Measure the motor's phase resistance at DC and its inductance from the current rise after a voltage step.
4. Close the current loops in the rotor frame with a forced angle (the angle generated by your code). Tune K_p and K_i from R, L and a crossover of 2 kHz, and capture an i_q step response.
5. Spin the motor in open loop with a rotating current vector, then implement the flux observer and a phase-locked loop, and hand over to closed-loop angle above 500 rpm. Add a speed PI loop.
6. Only now, build TI's universal motor control lab for the same hardware and compare its FAST or eSMO angle estimate with yours.

DONE WHEN

- ☐ The i_q step settles within 1 ms with under 10 % overshoot.
- ☐ The motor starts and enters closed loop on ten attempts out of ten, and holds speed setpoints from 500 to 3,000 rpm within 2 %.
- ☐ The ADC interrupt's execution time is measured and is under 60 % of the PWM period.

EVIDENCE FOR THE NOTEBOOK

- Register settings with their reasons; the R and L measurements; current-loop and speed-loop captures; the comparison with TI's estimator.

LAB 3.1.3 Your FOC against a commercial ESC

6 h

Goal. learn what a production ESC does better and worse than your code.

Kit. Lab 3.1.2's set-up, a SunnySky motor and prop on the thrust stand, a CloudPhoenix ESC, the 3S pack, the INA228 with the 1 mΩ shunt, safety glasses.

1. Re-identify R and L for the SunnySky motor and retune your current loops. Raise the PWM frequency to 40 kHz and check that the control step still fits.

2. Run your FOC on the thrust stand from the pack with a firmware current limit of 10 A. Repeat the thrust sweep of Lab 3.1.1 to the highest throttle the limit allows.
3. Repeat the sweep on the same motor and prop with the CloudPhoenix. Compare grams per watt at four thrust levels, phase-current waveforms on the scope (through a current probe or the BoosterPack's sense output), acoustic noise on a phone's sound meter, and the thrust step response.

DONE WHEN

- ☐ Both sweeps are on one plot with efficiency differences quantified, and the notebook explains each difference from the waveforms.

EVIDENCE FOR THE NOTEBOOK

- The sweeps, waveform captures and the written comparison.

LAB 3.1.4 The IMU and its noise

6 h, plus a 3 h unattended log

Goal. characterise the sensor the drone's control depends on most.

Kit. MSP-EXP432E401Y, LSM6DSOX breakout, logic analyser, TMP117.

1. Wire the LSM6DSOX to SSI2 on the LaunchPad. Write a driver from the datasheet: check WHO_AM_I, set the gyro to ±1,000 °/s and the accelerometer to ±8 g at 1.66 kHz, enable the data-ready interrupt on INT1, and read the burst of twelve output bytes in one SPI transaction. Capture and annotate the transaction.
2. Log three hours with the board still and level, with a TMP117 taped beside it. Compute the Allan deviation of each gyro axis and read off angle random walk and bias instability. Compare with the datasheet's noise density.
3. Calibrate the accelerometer by holding it still in six orientations; fit offset and scale per axis. Fit gyro bias against temperature from the long log.

DONE WHEN

- ☐ The Allan deviation plot shows the $-\frac{1}{2}$ slope and a bias-instability floor, with both numbers extracted.
- ☐ Accelerometer calibration brings $|g|$ to within 0.5 % in all six orientations.

EVIDENCE FOR THE NOTEBOOK

- Driver source; annotated SPI capture; the Allan plot and fitted terms; calibration tables.

LAB 3.1.5 Attitude estimation

8 h

Goal. estimate attitude from the gyroscope's rates and the accelerometer's forces.

Kit. Lab 3.1.4's set-up, a protractor or printed angle scale, one SunnySky motor with ESC and prop on the thrust stand.

1. Implement a one-axis complementary filter at 1 kHz on the MCU and tune its crossover. Tilt the board through known angles against the scale and plot estimate against truth.
2. Implement a quaternion Mahony filter with gyro-bias estimation. Report the error after a 10-minute still period, and after a rapid 90° rotation and return.
3. Tape the IMU board to the thrust stand's base and run the motor at hover throttle. Log raw accelerometer data at 1.66 kHz and plot its spectrum. Find the motor's fundamental and harmonics, then set the sensor's on-chip filters and your own notch so that the estimator's error with the motor running is within 1° of the error with it stopped.

DONE WHEN

- ☐ Static angle error under 1° and the error after the 90° manoeuvre under 2°.
- ☐ The vibration spectrum is annotated and the chosen filters are justified from it.

EVIDENCE FOR THE NOTEBOOK
- Estimator source; angle plots against the scale; the spectra before and after filtering.

LAB 3.1.6 Magnetometer calibration and heading — 5 h

Goal. measure an accurate heading on a frame that carries motor current.

Kit. LIS3MDL breakout, the LaunchPad, the frame with one motor and ESC fitted, the 3S pack, a phone compass for comparison.

1. Write the LIS3MDL driver. Collect about 2,000 samples while rotating the board through every orientation, fit an ellipsoid (offset and a 3 × 3 correction), and show the corrected samples on a sphere.
2. Mount the sensor on the frame and repeat. Compare the two calibrations.
3. Run one motor at four throttle levels and measure the field change at the sensor's position and at 5, 10 and 15 cm above the frame. Choose the mast height for Chapter 3.5 from the data.
4. Compute a tilt-compensated heading from the Mahony attitude and compare it with the phone at eight headings.

DONE WHEN
- ☐ Corrected samples lie within 3 % of a sphere, and heading agrees with the phone within 5° at all eight headings.
- ☐ The mast height is chosen from measured current interference.

EVIDENCE FOR THE NOTEBOOK
- Raw and corrected point clouds; the interference measurements; the heading table.

LAB 3.1.7 PID on a one-axis rig — 2 weekends

Goal. design a controller in simulation, then make it hold a real frame level.

Kit. The Tarot frame with two opposite motors, ESCs and props fitted, a wooden A-frame with a 10 mm steel rod through the frame's centre plates as a roll axle, the LaunchPad with the IMU, the 3S pack, a push button on a 1 m lead, safety glasses.

1. Build the rig so the frame can only roll, with mechanical stops at ±30°. Measure the frame's inertia about the axle with a swing test (a known offset mass and the oscillation period).
2. Write the one-axis model with the motor time constant from Lab 3.1.1 and the loop delays your firmware will have. In Python, design a rate loop and an angle loop to a stated phase margin, and simulate steps and a disturbance of a 50 g mass hung from one arm.
3. Drive the two ESCs with Oneshot125 from two timer PWM outputs at 1 kHz. Wire the push button to a GPIO as a dead man's switch: the motors run only while you hold it, and releasing it disarms within 50 ms. Chapter 3.5 replaces it with the radio link.
4. Implement the cascade with derivative on measurement, derivative filtering, integrator clamping and output saturation. Bring up the rate loop alone, then close the angle loop.
5. Compare measured and simulated step responses and disturbance rejection. Find the gain at which the rig oscillates and compare it with the model's prediction.

DONE WHEN
- ☐ The frame holds level within ±2° and recovers from the 50 g disturbance within 1 s.
- ☐ Measured and simulated step responses agree in rise time and overshoot within 30 %, or the notebook explains the model error.
- ☐ Releasing the dead man's switch disarms within 50 ms, tested ten times.

EVIDENCE FOR THE NOTEBOOK
- Rig photographs; the model and simulation code; step and disturbance plots, measured over simulated; the oscillation test.

Problem set

1. From your Lab 3.1.1 data, estimate hover current and flight time for a 1.2 kg drone on the 5,000 mAh pack using 80 % of capacity. How do both change if the drone gains 200 g?
2. A motor has $R = 0.13\ \Omega$ and $L = 30\ \mu\text{H}$ per phase. Choose the current-loop gains for a 3 kHz crossover and give the discrete gains at a 40 kHz control rate. What limits the crossover from above?
3. At 10,000 rpm with 7 pole pairs, what is the electrical frequency? How many PWM periods per electrical cycle at 20 kHz and at 45 kHz? Why does TI's drone ESC design use the higher rate?
4. Space-vector modulation reaches a phase-voltage amplitude of $V_{\text{dc}}/\sqrt{3}$, sinusoidal PWM only $V_{\text{dc}}/2$. Derive both, and give the speed headroom this buys on a 3S pack.
5. An IMU gyro has an angle random walk of 0.3 °/√h and a bias instability of 5 °/h. Estimate the attitude error after 10 s and after 10 minutes of pure integration, and say which term dominates each.
6. The motor fundamental at hover is 85 Hz and the IMU is sampled at 1 kHz with no anti-alias filter. Where does the 3rd harmonic alias if the sample rate drops to 200 Hz?
7. A complementary filter runs at 1 kHz with a 0.5 Hz crossover. Compute α. What steady-state angle error does a constant 1 m/s^2 sideways acceleration cause?
8. Your rate loop crosses over at 30 Hz with 50° of phase margin. The IMU's digital filter adds 2 ms of delay and the ESC protocol another 1 ms. What is the new margin, and what would you change first?
9. Derive the swing-test formula for the rig's inertia and estimate its uncertainty from a 2 % period error and a 5 % error in the offset distance.

Deliverables and stretch

Deliverables. The propulsion model (thrust, power and time constant against throttle) as a table and a script; the FOC firmware and the comparison report; the IMU, magnetometer and estimator drivers as a library that Chapter 3.5 links unchanged; the one-axis model, simulation and rig results.

Stretch. Add a second load cell to measure propeller torque and fit k_Q. Implement bidirectional DShot on your FOC firmware so it reports electrical rpm, and replace the CloudPhoenix on one arm with it. Implement an extended Kalman filter for attitude with gyro-bias states and compare it with Mahony on the same logs. Identify the rig's plant by injecting a chirp into the rate setpoint and fitting a transfer function.

COMPLETION CHECKLIST

- ☐ **Lab 3.1.1:** calibrated stand; k_T, grams per watt at hover and the thrust time constant measured.
- ☐ **Lab 3.1.2:** your FOC holds speed from 500 to 3,000 rpm and starts ten times out of ten; current step within spec.
- ☐ **Lab 3.1.3:** your FOC and the commercial ESC compared on one plot, with the differences explained.
- ☐ **Lab 3.1.4:** IMU driver, Allan deviation with both noise terms, accelerometer calibration.
- ☐ **Lab 3.1.5:** complementary and Mahony filters within error limits; vibration filtered from a measured spectrum.
- ☐ **Lab 3.1.6:** magnetometer calibrated on the frame; mast height chosen from data.
- ☐ **Lab 3.1.7:** frame held level within ±2°; simulation and measurement compared; dead man's switch tested ten times.
- ☐ **Problem set:** all nine answered with units.
- ☐ **You can explain**, without notes: why thrust per watt falls with load, what each FOC block does, how a complementary filter splits the spectrum, what Allan deviation shows, and why delay costs phase margin.

CHAPTER 3.2

Radio links

TIME	3 weeks
YOU BUILD	Two CC1312R LaunchPads reworked for conducted measurement; a packet radio written directly on TI's RF driver; a measured packet-error waterfall at two data rates, with the test set-up's leakage floor quantified; the drone's telemetry protocol with a duty-cycle budget, a UART bridge and a ground station; a field range test against your Chapter 1.5 link budget; a software demodulator that decodes your own packets from an SDR capture.
NEEDS	Two LAUNCHXL-CC1312R1, the Linx 868 MHz antennas, the tinySA Ultra, the RTL-SDR, the Mini-Circuits attenuators (10, 20 and three 30 dB), SMA cables and adapters, a die-cast aluminium box with an SMA bulkhead and a USB power bank, the microscope and hot-air station, the Raspberry Pi 5, the Windows PC with SmartRF Studio 7, Code Composer Studio with the SimpleLink Low Power F2 SDK.

WHAT THIS BUYS OMH

A drone without telemetry is flown blind, and a fleet of server modules without a management network cannot be operated. Both depend on links whose loss rate, latency and capacity were designed and then measured. This chapter goes below every protocol stack to the packet on the air: what decides how far it reaches, how often it is lost, how long it occupies the channel, and how you prove each number on a bench. The habits carry over directly to OMH's management and data networks, where a report that "the link drops packets sometimes" should start an investigation.

SKIP TEST

1. A packet has a 4-byte preamble, a 4-byte sync word, a length byte, 32 bytes of payload and a 2-byte CRC. What is its airtime at 50 kbit/s, and how many can you send per hour within a 1 % duty cycle?
2. What is the modulation index of 2-GFSK at 50 kbit/s with 25 kHz deviation, and what does the Gaussian filter change about the spectrum?
3. Two radios each have a ±20 ppm crystal. What is the worst-case frequency offset at 868 MHz, and why does it matter for a receiver with an 87 kHz filter?
4. Why does a packet-error-rate curve fall from 100 % to 1 % over only a few decibels?
5. You need to test a receiver at −110 dBm with a transmitter whose lowest setting is −20 dBm. What attenuation do you need, and what else will spoil the measurement?

If all five are easy, do Labs 3.2.3 and 3.2.4 only.

Core ideas

Modulation for low-power links. The CC1312R, like most sub-gigahertz radios, transmits frequency-shift keying: the carrier moves up by the deviation Δf for a one and down for a zero. The modulation index $h = 2\Delta f / R_b$ relates deviation to bit rate; $h = 1$ (25 kHz at 50 kbit/s) is a common choice. A Gaussian filter on the bit stream before modulation (GFSK) rounds the frequency transitions and narrows the spectrum at the cost of some intersymbol interference. Narrower filters and lower rates improve sensitivity. Chapter 1.5's $-174 + 10\log_{10} B + \mathrm{NF} + \mathrm{SNR}_{\min}$ applies directly, and TI's long-range mode runs at a twentieth of the bit rate to gain about 10 dB. ExpressLRS on the 2.4 GHz control link uses LoRa chirp spread spectrum or FLRC on Semtech radios, which rest on the same ideas.

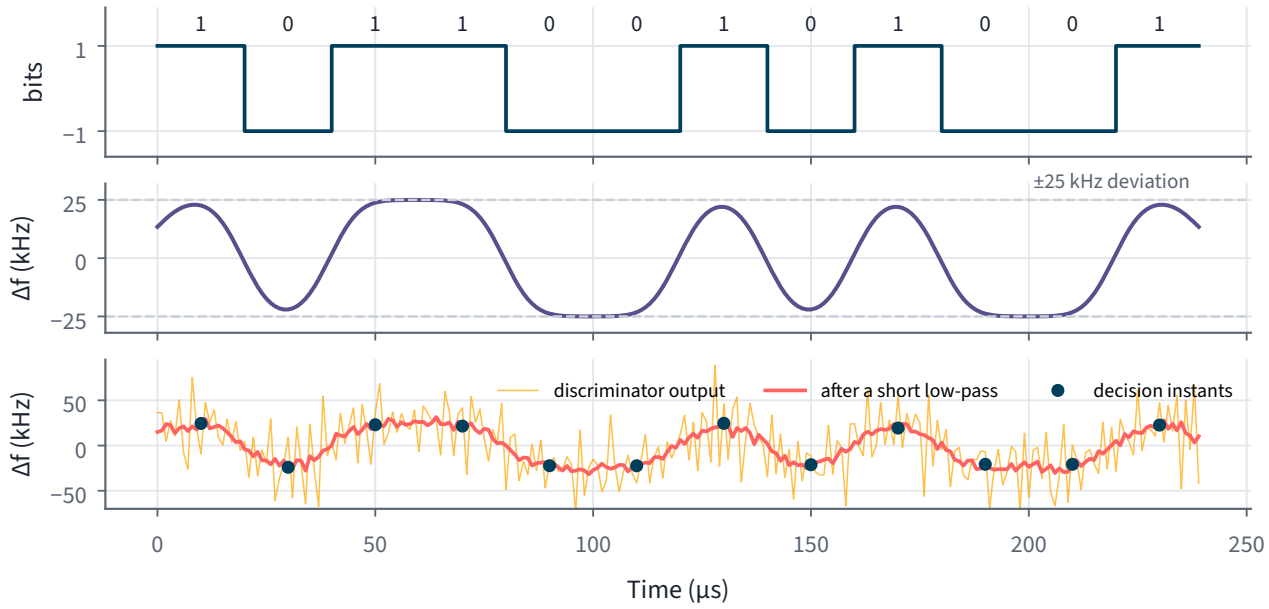


Figure 3.2.1. 2-GFSK: the bit stream, its Gaussian-filtered frequency, and what an FM discriminator recovers from a noisy capture. Lab 3.2.6 implements the bottom panel.

A packet on the air. A preamble of alternating bits lets the receiver settle its gain, estimate the frequency offset and lock its bit clock. A sync word marks the start of the frame; the receiver correlates against it and accepts a match within a set number of bit errors, which trades missed packets against false detections. A length field, the payload and a CRC follow. Whitening XORs the data with a pseudo-random sequence so long runs of identical bits do not upset the receiver's clock recovery. The CC1312R's radio core does all of this in hardware; you choose the parameters.

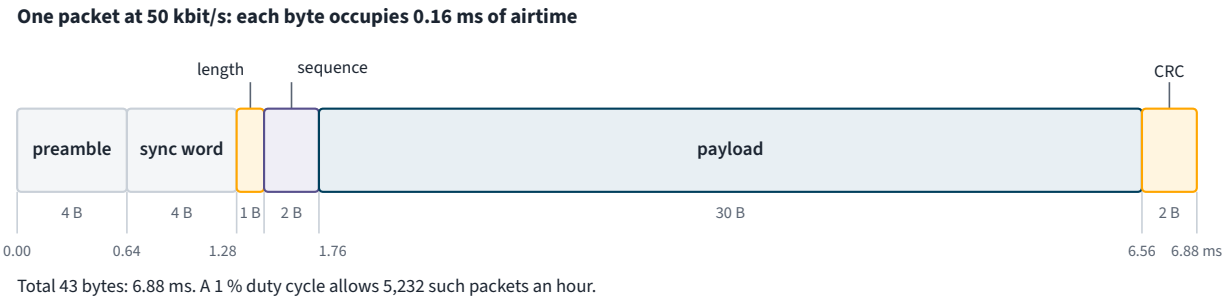


Figure 3.2.2. The packet format Lab 3.2.2 defines, with the airtime of each field at 50 kbit/s. The preamble and sync word are overhead you pay on every packet, which is why tiny packets waste airtime.

The CC1312R. An Arm Cortex-M4F application processor at 48 MHz runs your code. A separate radio core, a Cortex-M0 running TI's firmware, runs the modem and the packet engine. You talk to it by posting radio operation commands (set-up, frequency synthesiser, transmit, receive) through TI's RF driver; each command has a trigger, a start time on the 4 MHz radio timer, and a status the driver reports back. SysConfig generates the register settings for a chosen physical layer. The F2 SDK's EasyLink and TI 15.4-Stack sit on top; this chapter does not use them, because a drone's telemetry needs latency and duty-cycle control they hide.

Errors and the waterfall. For noncoherent binary FSK in white noise the bit error rate is $P_b = \frac{1}{2} e^{-E_b/2N_0}$, and a packet of N bits survives with probability $(1 - P_b)^N$. Because P_b falls exponentially with signal-to-noise ratio, the packet error rate drops from near 100 % to 1 % over a few decibels, a curve called the waterfall. Sensitivity is the received power at an agreed error rate and packet length. TI's measurement on this LaunchPad gives −110.5 dBm for 1 % bit error rate with 3-byte packets at 50 kbit/s, and −120.7 dBm in long-range mode (TI SWRA588A). Your measurement will use longer packets and so land higher, by an amount you can calculate.

Frequency error. Each radio's synthesiser is derived from a crystal; 20 ppm at 868 MHz is 17 kHz, and the two ends can be wrong in opposite directions. The receiver's channel filter must pass the signal plus the offset, and the demodulator must tolerate it. TCXOs, factory trimming and the receiver's offset estimate in the preamble are the remedies. A crystal's frequency also changes with temperature, so a drone's radio drifts in frequency as it climbs on a cold day.

Propagation beyond free space. Chapter 1.5's link budget assumed free space. Over the ground a reflected ray adds to the direct ray with a phase that depends on the heights and distance, producing nulls. Buildings and bodies add shadowing, and a vertical dipole has a null straight up, so the link can fail directly above the ground station. Polarisation mismatch between a tilted drone antenna and a vertical ground antenna adds further loss. The remedies are a fade margin of 15 to 20 dB, antenna placement, and diversity (two antennas, with the receiver using the better one).

Duty cycle as a design input. The 868 MHz short-range band is divided into sub-bands, each with a power limit and a maximum fraction of time a transmitter may occupy it, measured over an hour. Every product radio is designed to them, so your telemetry design starts from the table below. A 1 % duty cycle is 36 s of transmission per hour; at 50 kbit/s that is an average of 500 bit/s. Listen-before-talk with adaptive frequency agility is the alternative to a duty-cycle limit in some sub-bands.

Sub-band	Maximum power	Duty cycle
863.0–865.0 MHz	25 mW ERP	0.1 %
868.0–868.6 MHz	25 mW ERP	1 %, or listen-before-talk with frequency agility
868.7–869.2 MHz	25 mW ERP	0.1 %
869.40–869.65 MHz	500 mW ERP	10 %
869.7–870.0 MHz	5 mW ERP, or 25 mW ERP	none at 5 mW; 1 % at 25 mW

ERP is referenced to a half-wave dipole: ERP = EIRP − 2.15 dB. The CC1312R's maximum of +14 dBm (25 mW) into the Linx antenna fits every row; the 869.40–869.65 MHz sub-band's 10 % allowance is what makes a useful telemetry rate possible.

Protocol design. Telemetry is a stream where only the latest value matters: no acknowledgements, a sequence number to count losses, and fields packed in fixed point to the resolution the ground needs. Commands must arrive exactly once, so they carry a sequence number, are acknowledged, retried with a timeout, and made idempotent so a duplicate does no harm. Between the flight controller and the radio LaunchPad, a UART carries frames delimited by consistent-overhead byte stuffing (COBS) with a CRC, so a lost byte costs one frame and the receiver resynchronises at the next delimiter. The radio firmware enforces the duty-cycle budget itself with a token bucket, so no bug upstream can exceed it.

Measuring radios conducted. Over the air, the result depends on multipath and the room. Conducted measurement connects the transmitter's RF port through calibrated attenuators to the receiver's, so the received level is known to a decibel. The main pitfall is leakage. A board radiates from its traces and cables, and if that path delivers more signal than the attenuators do, the leakage sets the receiver's input level instead of the calibrated chain. The defences are a shielded box around the transmitter, a battery inside the box so no cable leaves it except the coaxial one, distance to the receiver, and a measured leakage floor.

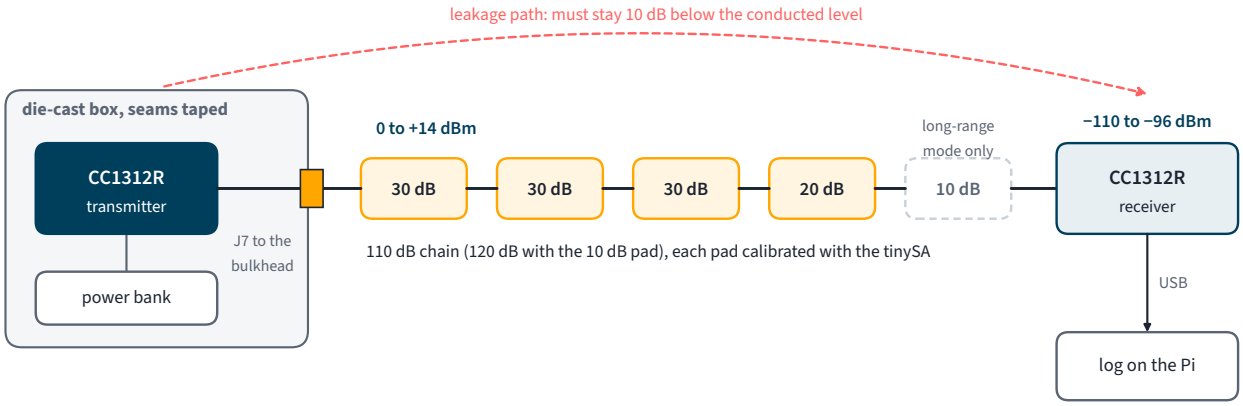


Figure 3.2.3. The conducted packet-error test of Lab 3.2.3. The dashed path is leakage; the test is valid only where the conducted level exceeds it by 10 dB or more, which you measure by terminating the chain.

Seeing your own signal. An SDR delivers complex baseband samples; the phase difference between consecutive samples is the instantaneous frequency, so an FM discriminator is one line of NumPy. Symbol timing, sync-word correlation, de-whitening and a CRC check turn that frequency trace back into your bytes. Decoding your own packets proves you understand every layer, and the same tools find the interferer that a spectrum analyser only shows as a bump.

Reading

- TI SWRA588A, *SimpleLink CC1312R LaunchPad for 868 MHz/915 MHz Bands*: the measured performance tables and section 5 on the RF design, including the SMA option.
- TI's CC13x2 Technical Reference Manual (SWCU185), the chapters on the radio, its command interface and the proprietary-mode packet format; the CC1312R datasheet's RF characteristics tables.
- The SimpleLink Low Power F2 SDK's RF driver API documentation and the `rfPacketTx`, `rfPacketRx` and `rfPacketErrorRate` examples.
- Proakis and Salehi, *Communication Systems Engineering*, the sections on FSK and noncoherent detection; or Sklar, *Digital Communications*, chapter 4.
- Cheshire and Baker, "Consistent Overhead Byte Stuffing" (IEEE/ACM ToN, 1999).
- Rappaport, *Wireless Communications*, chapter 4 (the two-ray model and link budgets).
- The tinySA Ultra's user guide sections on power measurement, the generator and maximum input.

Labs

SAFETY

A transmitter connected straight to an instrument can destroy it. Fit the 30 dB attenuator directly on the transmitter's SMA connector before anything else, every time, and write the expected level at the instrument in the notebook before you connect it. The tinySA's limit is 0 dBm for normal use.

LAB 3.2.1 Bring-up and the SMA rework — 6 h

Goal. get both radios working and make their RF ports measurable.

Kit. Two LAUNCHXL-CC1312R1, microscope, hot-air station, tweezers, flux, the tinySA with the 30 dB attenuator, the Windows PC with SmartRF Studio 7, CCS with the F2 SDK.

1. Build TI's `rfPacketTx` and `rfPacketRx` examples for the LaunchPad and flash one to each board. Confirm reception with the LEDs.

2. Read section 5.3 of SWRA588A: the RF path goes to the PCB antenna through C36, and moving that capacitor to the C37 pads routes it to SMA connector J7. Under the microscope, move it on both boards. Photograph before and after, and check with the multimeter that the capacitor's new pads are not shorted to ground.
3. With SmartRF Studio 7 driving one board in continuous-wave mode at 868.0 MHz, measure output power at J7 through the 30 dB attenuator for each power setting from the lowest to +14 dBm. Measure the second and third harmonics and the carrier frequency error.
4. Repeat the +14 dBm and frequency measurements on the second board.

DONE WHEN

- ☐ Both boards transmit through J7, and the power table agrees with the settings within ±1.5 dB after attenuator and cable loss.
- ☐ Harmonics and frequency error are measured for both boards.

EVIDENCE FOR THE NOTEBOOK

- Rework photographs; the power table; harmonic and frequency captures.

LAB 3.2.2 A packet radio below the stack — 8 h

Goal. write a packet radio on TI's RF driver and measure its timing to the microsecond.

Kit. Both LaunchPads with Linx antennas fitted, oscilloscope.

1. Write your own transmitter and receiver applications using only TI's RF driver and a SysConfig-generated 2-GFSK physical layer at 50 kbit/s with 25 kHz deviation. Define your packet: a 16-bit sequence number and a 30-byte payload, with whitening and the radio core's CRC.
2. Set a GPIO when you post the transmit command and clear it in the command-done callback. Compare the measured airtime with your calculation from the packet format, and measure the time from posting to the start of transmission.
3. On the receiver, count sequence gaps and CRC failures and read the RSSI of every packet; print statistics once a second over the back-channel UART.
4. Measure the receive-to-transmit turnaround: the receiver answers each packet, and you time the gap on the scope with both GPIOs.

DONE WHEN

- ☐ Measured airtime matches your calculation within 5 %, and the start latency and turnaround are measured.
- ☐ The receiver reports zero lost packets in 10,000 at 1 m.

EVIDENCE FOR THE NOTEBOOK

- Source; the airtime calculation; scope captures; the statistics log.

LAB 3.2.3 Sensitivity and the packet-error waterfall — 8 h

Goal. measure a receiver properly, including the limits of your own bench.

Kit. Both reworked LaunchPads, the die-cast box with an SMA bulkhead fitted and the USB power bank inside, the attenuators (three 30 dB, one 20 dB, one 10 dB), SMA cables, the tinySA, two 50 Ω terminators, copper tape.

1. Measure each attenuator and cable at 868 MHz with the tinySA's generator and input at levels well above its noise floor, and sum them to get the chain's loss.
2. Program the transmitter to step through its power settings from −20 to +14 dBm by itself, 1,000 packets at each, carrying the setting in the payload, because once the lid is on no cable except the coaxial one may leave the box.

3. Put the transmitter in the box on the power bank, its J7 to the bulkhead, and seal the lid's seam with copper tape. Outside, a 110 dB chain (three 30 dB and the 20 dB) runs to the receiver's J7, with the receiver a metre away on its own USB lead. The rework has already disconnected both boards' PCB antennas, which provides half of your isolation.
4. Leakage floor: disconnect the chain from the receiver and terminate both, then run the transmitter's sequence and record what the receiver still hears. This is your bench's floor.
5. Run the sequence through the chain, recording packet error rate and mean RSSI at each level. Plot packet error rate against received power for 50 kbit/s, then add the 10 dB attenuator and repeat for TI's long-range mode.
6. Compare your 1 % points with SWRA588A's figures, correcting for your longer packet using the bit-error model.

DONE WHEN

- ☐ Both waterfalls are plotted, and the leakage floor is at least 10 dB below every level you report, or the notebook shows how you improved the isolation until it was.
- ☐ Your 1 % packet-error points agree with TI's sensitivity within 3 dB after the packet-length correction.

EVIDENCE FOR THE NOTEBOOK

- Attenuator calibration; the leakage measurements; both waterfalls; the comparison.

LAB 3.2.4 The telemetry protocol

2 weekends

Goal. design a protocol from requirements and budgets, then build it end to end.

Kit. Both LaunchPads, the MSP-EXP432E401Y as a stand-in flight controller, the Raspberry Pi 5.

1. List what the ground needs from the drone (attitude, rates, altitude estimates, battery voltage, current and used capacity, control-link quality, arming state, faults) with the rate and resolution of each, and the commands the ground may send.
2. Choose the 869.40–869.65 MHz sub-band and compute the airtime budget at 10 % duty cycle. Pack the fields in fixed point and choose a packet rate that fits with 30 % margin.
3. Write the radio bridge: COBS-framed, CRC-checked frames arrive over UART from the flight controller and leave over the air; commands come back with acknowledgement, retry and sequence numbers. Enforce the duty-cycle budget with a token bucket in the radio firmware, and test it by flooding the bridge.
4. Write the ground side: the second LaunchPad forwards frames over USB serial to the Pi, which decodes, logs to CSV and plots live.
5. Measure end-to-end latency from the flight controller's frame to the Pi's decode, and command round-trip time.

DONE WHEN

- ☐ Telemetry streams at the designed rate, and the measured transmit duty cycle over 10 minutes is within the budget, including under flooding.
- ☐ Latency and command round trip are measured, and a dropped command is retried and executed once.
- ☐ A two-page protocol specification exists that another engineer could implement from.

EVIDENCE FOR THE NOTEBOOK

- The specification; the budget calculation; source; the duty-cycle and latency measurements.

LAB 3.2.5 Range in the field — 5 h outdoors

Goal. compare your link budget with the range you measure outdoors.

Kit. Both LaunchPads with whip antennas on power banks, a 1.5 m pole or tripod for the ground antenna, the Tarot frame, a phone for positions.

1. Fix the ground station at 1.5 m. Walk away along a clear path with the other radio, logging RSSI and packet error rate with positions every 25 m, until the error rate exceeds 10 %. Plot received power against distance over your Lab 1.5.6 budget.
2. Mount the radio on the drone frame (motors off) at the planned position and rotate the frame in 45° steps at a fixed distance. Plot the pattern and find the nulls.
3. Raise the remote radio well above the ground station (a window or a hill) and record what happens near the vertical.

DONE WHEN

- ☐ Measured and predicted received power are on one plot, with the discrepancy explained in terms of ground reflection, obstruction and antenna orientation.
- ☐ The frame-mounted pattern is measured and the radio's position on the drone is decided from it.

EVIDENCE FOR THE NOTEBOOK

- Logs with positions; the plots; photographs of the set-ups.

LAB 3.2.6 Demodulate your own packets — 6 h

Goal. decode your own packets from SDR samples in NumPy.

Kit. RTL-SDR, the transmitting LaunchPad with the 30 dB attenuator into a short antenna on the SDR, Python with NumPy and SciPy.

1. Capture 2 s of your 50 kbit/s packets at 1.024 MS/s centred 100 kHz below the carrier (to keep the dongle's DC spike out of the signal).
2. Shift to baseband, low-pass, and apply an FM discriminator. Measure the deviation and the frequency offset from the trace.
3. Recover symbol timing, correlate for your sync word, slice the bits, undo the whitening using the sequence in TI's documentation, and check the CRC.

DONE WHEN

- ☐ At least 99 of 100 strong packets decode with a correct CRC, and the measured deviation is within 10 % of 25 kHz.

EVIDENCE FOR THE NOTEBOOK

- The demodulator notebook with plots at each stage; the decoded payloads against what was sent.

Problem set

1. Compute the airtime of your Lab 3.2.2 packet at 50 kbit/s and in long-range mode, and the maximum packet rates in the 868.0–868.6 MHz and 869.40–869.65 MHz sub-bands.
2. The receiver's channel filter is 87 kHz wide and its noise figure 7 dB. Estimate its sensitivity for an SNR requirement of 9 dB and compare with −110.5 dBm.
3. Using the noncoherent FSK formula, find the E_b/N_0 for a 1 % packet error rate with 3-byte and with 34-byte packets. How many decibels separate them?
4. Two ±20 ppm crystals: give the worst-case offset at 869.5 MHz and the fraction of an 87 kHz filter it consumes. What would a ±2 ppm TCXO change?

5. A drone flies at 60 m altitude, 400 m horizontally from a ground antenna at 1.5 m over flat ground with reflection coefficient −1. Compute the path difference, the phase difference and the received power relative to free space.
6. An ideal half-wave dipole has the pattern $\cos\left(\frac{\pi}{2}\cos\theta\right)/\sin\theta$, with θ measured from its axis. Compute its relative gain at 30°, 60° and 80° from broadside. What does this mean for a vertical ground antenna and a drone overhead?
7. Design a token bucket that enforces 10 % over any hour but lets a 20 s burst through at the start of a flight. Give the bucket size and fill rate, and the worst case it permits.
8. A 40-byte telemetry frame crosses a 115,200 baud UART with COBS framing and a 2-byte CRC. What is the worst-case overhead, and the maximum frame rate?
9. Your chain has 110 dB of attenuation and the transmitter runs at 0 dBm. How much isolation must the leakage path have for a valid measurement at the 1 % point with 10 dB of margin? List three things that raise the isolation and estimate what each is worth.

Deliverables and stretch

Deliverables. The reworked boards; the packet-radio firmware; the waterfall measurements with the leakage floor; the telemetry protocol specification, bridge firmware and ground station; the field-test report; the demodulator.

Stretch. Add listen-before-talk with frequency agility in the 868.0–868.6 MHz sub-band and measure the throughput gain over the plain duty cycle. Add forward error correction (a Hamming or convolutional code) to your packets and measure the waterfall's shift. Look at the ExpressLRS 2.4 GHz link on the tinySA Ultra's high band and identify its hop pattern. Flash the BeaglePlay's CC1352P7 as a Linux-attached ground station.

COMPLETION CHECKLIST

- ☐ **Lab 3.2.1:** both boards reworked to J7; power table within ±1.5 dB; harmonics and frequency error measured.
- ☐ **Lab 3.2.2:** own packet radio; airtime within 5 % of calculation; 10,000 packets at 1 m with no loss.
- ☐ **Lab 3.2.3:** two waterfalls with a measured leakage floor; 1 % points within 3 dB of TI's after correction.
- ☐ **Lab 3.2.4:** telemetry at the designed rate within the duty-cycle budget; latency measured; specification written.
- ☐ **Lab 3.2.5:** field data plotted over the link budget; antenna position on the drone decided.
- ☐ **Lab 3.2.6:** 99 of 100 packets decoded from an SDR capture.
- ☐ **Problem set:** all nine answered with units.
- ☐ **You can explain**, without notes: what each field of a packet is for, why the error-rate curve is a waterfall, how you would prove a receiver's sensitivity, how a duty cycle shapes a protocol, and why commands and telemetry need different reliability.

CHAPTER 3.3

Microwave synthesis and FMCW radar

TIME	3 weeks
YOU BUILD	A TI frequency synthesiser programmed from your own register calculator over SPI from the Pi; measured lock time, loop dynamics and fractional spurs; a frequency chirp generated and measured sample by sample; an FMCW radar simulator written from first principles; a configured 60 GHz TI radar measuring range as an altimeter, read over UART by your own parser; raw radar samples captured and processed with your simulator's signal chain.
NEEDS	LMX2572EVM with its Reference PRO board, the bench supply with an SMA power lead, the IWRL6432BOOST, the FTDI C232HM cable, the tinySA, the RTL-SDR, the 10 and 30 dB attenuators and 50 Ω terminators, the oscilloscope, the Raspberry Pi 5, the Windows PC with TICS Pro, UniFlash and MMWAVE-L-SDK, a tape measure.

WHAT THIS BUYS OMH

Every high-speed interface in a server begins at a phase-locked loop: PCIe, Ethernet SERDES, DDR and the CPU's own clocks are all synthesised from a crystal, and their jitter budgets are phase-noise budgets stated in the time domain. Microwave synthesis shows the subject most clearly, because you can see the spectrum on an analyser. In a radar, a chirp from a synthesiser, mixed with its own echo, gives range as a frequency, and TI's 60 GHz radar-on-a-chip does this at milliwatts. The drone uses one as its altimeter in Chapter 3.5. You will understand every number in its configuration because you will have simulated the whole chain first.

SKIP TEST

1. A fractional-N synthesiser has a 100 MHz phase-detector frequency and a VCO range of 3.2 to 6.4 GHz. Give a divider setting and an output divider that produce 868.000 MHz.
2. Where does an integer-boundary spur appear when the VCO runs at 3,500.1 MHz with a 100 MHz phase detector and the output is the VCO divided by four?
3. An FMCW radar sweeps 4 GHz in 40 µs. What beat frequency does a target at 10 m produce, and what is the range resolution?
4. How does an FMCW radar measure velocity, and what sets the maximum unambiguous velocity?
5. Why does a phase-noise skirt on a spectrum analyser not tell you a synthesiser's phase noise unless the analyser is much better than the source?

If all five are easy, do Labs 3.3.4 and 3.3.5 only.

Core ideas

The phase-locked loop. A voltage-controlled oscillator (VCO) is free-running and drifts; a crystal reference is stable but fixed. A PLL divides the VCO's output by N, compares its phase with the reference (after an R divider) at the phase-frequency detector, and drives the VCO's tuning voltage through a charge pump and a loop filter until the two agree. In lock, $f_{\mathrm{VCO}} = N f_{\mathrm{PD}}$. An output (channel) divider then reaches frequencies below the VCO's range: the LMX2572's VCO covers 3.2 to 6.4 GHz, and its divider takes the output down to 12.5 MHz.

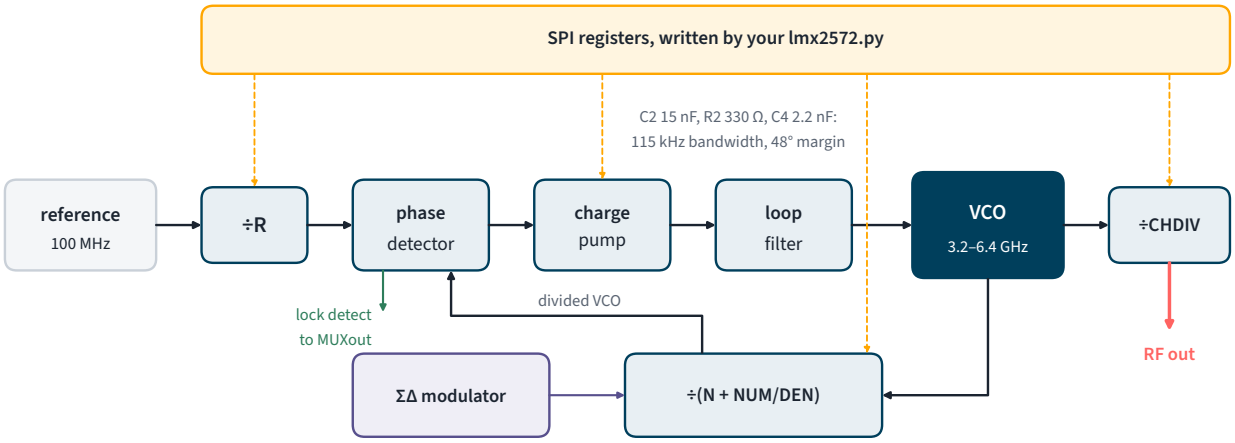


Figure 3.3.1. The LMX2572 as a block diagram. Lab 3.3.1 computes every number in this picture, writes it over SPI and checks the result on the spectrum analyser.

Fractional-N. With an integer N, the output step is f_{PD}, which is too coarse at 100 MHz. A fractional-N synthesiser switches N between neighbouring integers with a delta-sigma modulator so that its average is $N + \mathrm{NUM}/\mathrm{DEN}$; with a 32-bit denominator the step is a fraction of a hertz. Fractional-N adds noise and spurs. Quantisation noise is pushed to high offsets, where the loop filter removes it. Spurs appear where the fraction is nearly an integer: an integer-boundary spur sits at the distance between f_{VCO} and the nearest multiple of f_{PD}, divided by the output divider, and only the loop filter attenuates it. Moving f_{PD} or the VCO frequency moves the spur.

Loop dynamics. The charge-pump current, the VCO gain (66 MHz/V on this EVM) and the loop filter set the loop bandwidth and phase margin; the EVM is designed for 115 kHz and 48°. Inside the bandwidth the output follows the reference and phase-detector noise multiplied by N ($20 \log_{10} N$ in decibels); outside it the VCO's own noise dominates. Wider loops lock faster and suppress more VCO noise but pass more reference noise and spurs. Lock time is the VCO calibration (choosing a band and capacitor setting) plus the loop's analogue settling, which you can watch on the tuning voltage.

Phase noise and jitter. A real oscillator's spectrum has a skirt around the carrier, described by the single-sideband phase noise $\mathscr{L}(f)$ in dBc/Hz at offset f. Integrated over the offsets that matter, it gives RMS jitter, which is how PCIe and Ethernet budgets state it. Measuring it needs an instrument cleaner than the source. The tinySA's own local oscillator is far noisier than the LMX2572, so on the tinySA you see the analyser's phase noise. Every measurement includes the instrument, and on this bench that lesson matters more than the phase-noise figure.

FMCW radar. A frequency-modulated continuous-wave radar transmits a chirp whose frequency rises linearly with slope $S = B/T_c$ over bandwidth B. The echo from a target at range R returns after $\tau = 2R/c$ and, mixed with the chirp still being transmitted, produces a beat at $f_b = 2RS/c$. A Fourier transform of the sampled beat signal over one chirp is a range profile. Range resolution is $\Delta R = c/2B$, about 3.75 cm for 4 GHz. The maximum range is set by the highest beat frequency the receiver's IF filter and ADC pass: $R_{\max} = f_{\mathrm{IF,max}} c/2S$.

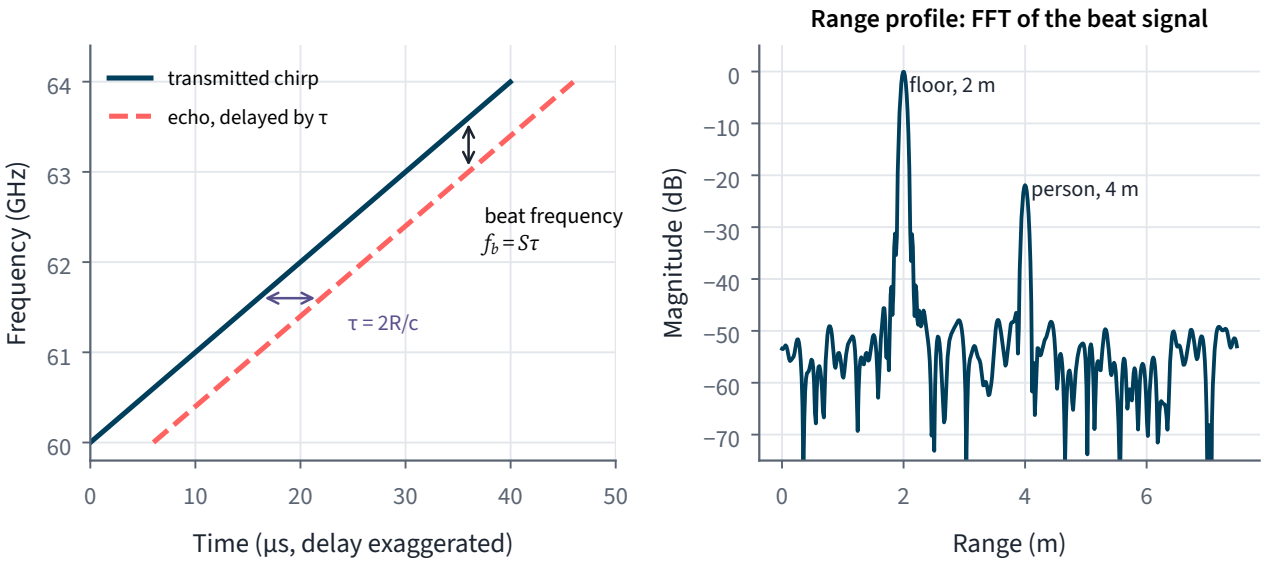


Figure 3.3.2. FMCW in one picture: the transmitted and received chirps, the beat frequency between them, and the range profile its Fourier transform produces. Lab 3.3.4 simulates this chain.

Velocity and angle. A moving target changes the echo's phase from chirp to chirp by $\Delta\varphi = 4\pi v T_c/\lambda$, so a second Fourier transform across chirps, at each range bin, gives velocity: the range-Doppler map. The maximum unambiguous velocity is $\lambda/4T_c$ and the velocity resolution $\lambda/2T_f$ for a frame of length T_f. With several receive antennas spaced $\lambda/2$ apart, the echo's phase across antennas gives its angle, and a third transform over antennas gives an angle spectrum. With two transmitters taking turns (TDM-MIMO), two transmit and three receive antennas act as a six-element virtual array.

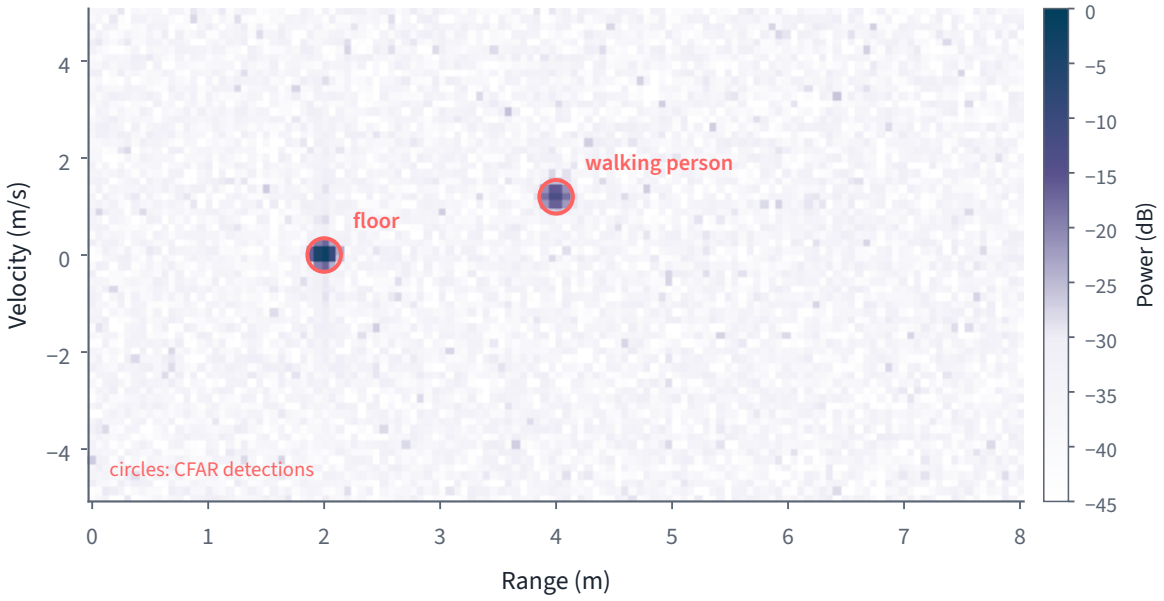


Figure 3.3.3. A simulated range-Doppler map for the altimeter configuration: the floor as a strong stationary return and a walking person at 4 m. A CFAR detector marks cells that rise above their local noise estimate.

Detection. A constant-false-alarm-rate (CFAR) detector compares each cell with the average of its neighbours, excluding a guard band, and declares a detection when it exceeds that average by a set factor. It adapts to clutter and noise floors that vary across the map. Windowing before each transform trades resolution for sidelobe level: a Hann window widens the main lobe but drops the first sidelobe from −13 dB to −31 dB, which keeps a strong floor return from hiding a weak target beside it.

The radar equation and the ground. For a point target received power falls as $1/R^4$; for an extended surface filling the beam, such as the ground beneath a drone, it falls closer to $1/R^2$, which makes altimetry an easy task for radar. Specular surfaces (calm water, polished floors) reflect away from the radar when it tilts;

rough ground scatters back. On a drone the beam also sees the propellers, whose blades produce micro-Doppler sidebands, so mounting and range gating matter.

TI's IWRL6432. The IWRL6432 is a 57 to 64 GHz FMCW radar-on-a-chip with two transmitters, three receivers, the chirp synthesiser, mixers, IF filters and ADCs, a hardware FFT accelerator and an Arm Cortex-M4F, designed for low-power sensing. The BoosterPack adds the antennas, an XDS110 debugger and a USB connection that powers the board and carries a UART. TI's MMWAVE-L-SDK provides a demo that runs the full chain on the chip and reports results over the UART as frames of type-length-value records; you configure it with text commands that set the chirp, the frame and the processing. Its raw ADC samples can also be streamed out over SPI, which Lab 3.3.6 uses to check the chip's processing against your own.

Reading

- TI SNAU217B, *LMX2572EVM Evaluation Instructions*, all of it; the LMX2572 datasheet's sections on the PLL, VCO calibration, the channel divider, ramping, the register map and the recommended power-up sequence.
- Banerjee, *PLL Performance, Simulation, and Design* (free from TI), chapters on loop filters, phase noise, spurs and lock time. Use TI's PLLatinum Sim tool alongside it.
- TI SPYY005, *The Fundamentals of Millimeter Wave Radar Sensors*; TI SWRA553, *Programming Chirp Parameters in TI Radar Devices*; and the "Introduction to mmWave Sensing: FMCW Radars" training series on TI's website.
- Richards, *Fundamentals of Radar Signal Processing*, 2nd ed., the chapters on the radar equation, waveforms and range processing, Doppler processing and CFAR detection.
- TI SWRU596, the IWRL6432BOOST user's guide: switch settings, flashing, and the BoosterPack and DCA1000 connectors. The MMWAVE-L-SDK documentation for the motion and presence demo's command-line interface and output format, and for SPI ADC streaming.

Labs

SAFETY

The LMX2572EVM takes 3.0 to 3.6 V on its VCC connector: set the bench supply to 3.30 V with a 200 mA limit before connecting it, and check the polarity of your power lead with the meter. Terminate every unused RF output with 50 Ω. Fit the 10 dB attenuator on the tinySA input for every synthesiser measurement.

LAB 3.3.1 A PLL you program — 8 h

Goal. program the synthesiser over SPI from your own register calculator instead of the vendor's GUI.

Kit. LMX2572EVM, Reference PRO, bench supply with an SMA power lead, tinySA with the 10 dB attenuator, two 50 Ω terminators, Raspberry Pi 5, jumper wires, the Windows PC with TICS Pro.

1. Set up the EVM as SNAU217B section 2 describes: 3.3 V on VCC, the Reference PRO's 100 MHz output on OSCinP through the SMA adaptor with its other output terminated, RFoutAP through the attenuator to the tinySA and RFoutAM terminated. In TICS Pro, load the default mode and write all registers. Confirm 3 GHz on the tinySA and the lock LED.
2. Export TICS Pro's register file. Then remove the ribbon cable, keep the Reference PRO powered from a USB charger, and wire the Pi's SPI0 (SCLK, MOSI, CE0) and ground to the EVM's SCK, SDI and CSB test points, with a GPIO to CE and MUXout to a GPIO input. Set MUXout_SW switch 2 to Break so MUXout can carry readback.
3. Write `lmx2572.py`: from the datasheet's register map, compute the VCO frequency, channel divider, `PLL_N`, `PLL_NUM` and `PLL_DEN` for a requested output, patch those fields into TICS Pro's baseline, and program the device using the datasheet's power-up sequence. Read registers back over MUXout.

4. Lock at 868.000 MHz and 2,440.000 MHz. Check frequency and level on the tinySA and lock detect on MUXout. Then generate the same two frequencies in TICS Pro and diff its registers against yours.

DONE WHEN

- [ ] Your code locks at both frequencies with lock detect asserted, and the tinySA confirms each within its frequency accuracy.
- [ ] Every register difference from TICS Pro's export is explained in the notebook.

EVIDENCE FOR THE NOTEBOOK

- The calculator source; the register diffs; tinySA captures; the readback log.

LAB 3.3.2 Loop dynamics, spurs and lock time — 6 h

Goal. measure the loop's settling, and find the spurs a fractional-N synthesiser makes.

Kit. Lab 3.3.1's set-up, oscilloscope with a 10× probe on the Vtune test point.

1. Lock time: alternate between 868 and 900 MHz from your code. Trigger the scope on CSB's last rising edge and capture MUXout's lock detect and the tuning voltage. Separate VCO calibration time from analogue settling.
2. Loop dynamics: make a small frequency step that needs no VCO calibration and capture the tuning voltage's settling. Estimate the natural frequency and damping, and compare with the 115 kHz and 48° design. Reduce the charge-pump current to a quarter and repeat.
3. Spurs: set the VCO to 3,500.1, 3,500.5 and 3,502 MHz with the output divider at 4, and find the integer-boundary spur at the output in each case. Tabulate its offset and level, then move it by changing the phase-detector frequency.
4. Phase noise: capture the noise skirt of the LMX2572 at 868 MHz and of the tinySA's own generator at the same frequency and settings. Explain why they look alike.

DONE WHEN

- [ ] Lock time is split into calibration and settling, and the loop's natural frequency is measured at two charge-pump currents with the change explained.
- [ ] The spur offsets match your prediction at all three settings.

EVIDENCE FOR THE NOTEBOOK

- Scope captures; the settling fits; the spur table; the two phase-noise captures with the explanation.

LAB 3.3.3 A chirp you can measure — 6 h

Goal. generate a frequency ramp and measure its linearity sample by sample.

Kit. Lab 3.3.1's set-up, RTL-SDR, the 30 dB and 10 dB attenuators.

1. Configure the LMX2572's calibration-free automatic ramp from SNAU217B section 3.2.2 (`OUT_FORCE`, `LD_DLY` and `PLL_DEN` as stated there), scaled so the output sweeps 2 MHz around 868 MHz in a triangle with 1 ms up and 1 ms down.
2. Feed the output through 40 dB of attenuation to the RTL-SDR and capture at 2.4 MS/s centred on the ramp.
3. Compute instantaneous frequency from the phase difference between samples. Fit the up and down slopes, plot the residual from a straight line, and measure what happens at each turnaround.

DONE WHEN

- [ ] Measured slopes are within 1 % of the programmed slope, and linearity error and turnaround behaviour are quantified.

EVIDENCE FOR THE NOTEBOOK
- Register settings; the capture; frequency-against-time and residual plots.

LAB 3.3.4 FMCW in simulation 8 h

Goal. build the whole radar signal chain in simulation before you use the radar.

Kit. Python with NumPy, SciPy and Matplotlib.

1. Write a simulator: chirp parameters (start frequency, slope, ADC rate and samples, chirps per frame, idle time), point targets with range, velocity and angle, and an extended floor; generate the complex beat signal at each of six virtual antennas with noise.
2. Process it: windowed range FFT, Doppler FFT, a cell-averaging CFAR detector, and an angle FFT for each detection. Show the range-Doppler map and a detection list.
3. Design the altimeter: range resolution under 5 cm, maximum range at least 15 m, 50 frames per second, and the lowest active duty cycle that meets them. Read the IWRL6432's limits (bandwidth, slope, IF bandwidth, ADC rate) from its datasheet, and show each requirement met with the arithmetic.
4. Simulate the drone tilted by 20° over flat ground and plot the reported altitude against true height.

DONE WHEN
- ☐ Simulated targets appear at the right range, velocity and angle within one bin, and the CFAR detects them with no false alarms in a noise-only run.
- ☐ The altimeter configuration meets every requirement, with the tilt error quantified.

EVIDENCE FOR THE NOTEBOOK
- The simulator; maps and detection lists; the configuration with its arithmetic.

LAB 3.3.5 The IWRL6432 as an altimeter 8 h

Goal. configure a real radar from your design and measure it against a tape measure.

Kit. IWRL6432BOOST, micro-USB cable, the Windows PC with UniFlash and the SDK's visualiser, the Raspberry Pi 5, the INA228, tape measure, a flat board as a target.

1. Set the SOP switches for flashing as SWRU596 describes, flash the motion and presence demo from MMWAVE-L-SDK with UniFlash, return the switches to functional mode and run TI's visualiser to confirm it works.
2. On the Pi, write a configuration loader that sends your Lab 3.3.4 configuration over the board's UART, and a parser for the demo's output frames (a magic word, a header and type-length-value records). Extract the range profile and the detections.
3. Point the radar at a wall and then down at the floor. Measure range from 0.3 to 5 m in ten steps against the tape measure, 100 frames at each, and report bias and spread.
4. Measure the board's current on its 5 V supply with the INA228 for the default configuration and yours, and compare with the duty-cycle estimate from your design.

DONE WHEN
- ☐ Range error is under 3 cm RMS between 0.5 and 5 m.
- ☐ Measured power agrees with your duty-cycle estimate within 30 %, or the discrepancy is explained.

EVIDENCE FOR THE NOTEBOOK
- Loader and parser sources; the configuration; the range table; the power measurements.

LAB 3.3.6 Raw ADC data 6 h

Goal. check the chip's processing against your own, from the same samples.

Kit. IWRL6432BOOST, FTDI C232HM-DDHSL-0 cable, the Windows PC with MMWAVE-L-SDK, Python with `pyftdi` on the Pi or your workstation.

1. Rebuild the demo with SPI ADC streaming enabled, as the SDK's documentation describes, and connect the FTDI cable to the pins it names.
2. Capture raw ADC frames with TI's receiver tool on the Windows PC. Then write your own receiver with `pyftdi` and confirm it captures identical frames.
3. Process the frames with your Lab 3.3.4 pipeline and compare your range profile with the one the chip reports for the same frame. Measure the floor return's signal-to-noise ratio.

DONE WHEN

- [ ] Your range profile's peak matches the chip's within one bin on ten consecutive frames, and the SNR of the floor return is measured.

EVIDENCE FOR THE NOTEBOOK

- Your receiver; the comparison plots; the SNR measurement.

Problem set

1. With $f_{\mathrm{PD}} = 100$ MHz and `PLL_DEN` $= 2^{32} - 1$, give the channel divider, `PLL_N` and `PLL_NUM` for 868.000 MHz and for 2,440.000 MHz. What is the frequency step at each output?
2. For an output of 875.03 MHz with the channel divider at 4 and $f_{\mathrm{PD}} = 100$ MHz, where is the integer-boundary spur? Choose a different phase-detector frequency that moves it beyond the loop bandwidth.
3. In-band phase noise is approximately $\mathscr{L}_{\mathrm{norm}} + 10\log_{10} f_{\mathrm{PD}} + 20\log_{10} N$. With $\mathscr{L}_{\mathrm{norm}} = -229$ dBc/Hz, compare 868 MHz synthesised with $f_{\mathrm{PD}} = 100$ MHz and with $f_{\mathrm{PD}} = 25$ MHz. Why do designers like high phase-detector frequencies?
4. A chirp sweeps 3 GHz in 30 µs. Compute the slope, the beat frequency at 2 m and at 15 m, the range resolution, and the maximum range if the IF bandwidth is 5 MHz.
5. At 60 GHz with a 120 µs chirp period, what is the maximum unambiguous velocity? With 32 chirps per frame, what is the velocity resolution?
6. Six virtual antennas spaced $\lambda/2$ apart: what is the angular resolution at broadside, and why do you get ambiguous angles if the spacing exceeds $\lambda/2$?
7. Explain why the floor return falls as roughly $1/R^2$ when the floor fills the beam. By how many decibels does it fall from 1 m to 10 m, and how does that compare with a point target?
8. The drone tilts 15° with a radar whose beam is wide. What range does the strongest return report for a height of 5 m over flat rough ground, and over calm water?
9. A strong signal sits 100 kHz from a weak wanted signal. If the receiver's local oscillator has −100 dBc/Hz of phase noise at 100 kHz offset, how much noise does reciprocal mixing add in a 50 kHz channel, relative to the strong signal?

Deliverables and stretch

Deliverables. `lmx2572.py` with its register diffs; the loop, spur and lock-time measurements; the chirp measurement; the FMCW simulator with the altimeter design; the radar loader, parser and range measurements; the raw-ADC receiver and comparison.

Stretch. Use the LMX2572's FSK mode to transmit packets your Lab 3.2.6 demodulator decodes. Characterise the IWRL6432's range accuracy against temperature using the TMP117. Add a Kalman tracker to the simulator and the parser that follows a walking person. Reproduce the chirp measurement for a 10 MHz ramp at 2.44 GHz with the tinySA's zero-span mode.

COMPLETION CHECKLIST

- ☐ **Lab 3.3.1:** your calculator locks at 868 MHz and 2.44 GHz; every difference from TICS Pro explained.
- ☐ **Lab 3.3.2:** lock time split; loop dynamics at two charge-pump currents; spur offsets predicted; the phase-noise lesson written down.
- ☐ **Lab 3.3.3:** chirp slope within 1 %; linearity and turnaround measured.
- ☐ **Lab 3.3.4:** simulator with range, Doppler, CFAR and angle; altimeter configuration meets its requirements.
- ☐ **Lab 3.3.5:** range error under 3 cm RMS from 0.5 to 5 m; power measured against the estimate.
- ☐ **Lab 3.3.6:** your processing matches the chip's within one range bin.
- ☐ **Problem set:** all nine answered with units.
- ☐ **You can explain**, without notes: how a fractional-N PLL makes a frequency and why it makes spurs, what sets loop bandwidth, how a chirp turns range into frequency, and what each FFT in a radar chain is for.

CHAPTER 3.4

Board design and bring-up

TIME	6 weeks, plus two fabrication lead times of 2 to 3 weeks each
YOU BUILD	Four printed circuit boards, specified, designed in KiCad, reviewed, fabricated, assembled and brought up: Board P, the drone's power module; Board F, the flight-controller BoosterPack; Board A, a BMC-lite management controller built on the MSP432E401Y chip; and Board B, a Compute Module 5 carrier with NVMe and a TPM. An errata list and a respin decision for each.
NEEDS	KiCad 9 or later, the bench kit (supply, scope, multimeter, microscope, hot-air station, thermal camera if you have one), the Raspberry Pi Debug Probe, the USB-to-serial adapters, your firmware from Chapters 2.6 and 3.1, your boot chain from Chapter 2.7, an account with a PCB fabricator that also assembles (JLCPCB is assumed below), the CM5 module and a spare NVMe drive for Board B.

WHAT THIS BUYS OMH

OMH sells hardware. Until now the course has run on other people's boards. From here the boards are your own designs, and you pay for their mistakes at fabrication. Four boards teach the cycle a hardware company works in: specify, draw the schematic, lay out, review, fabricate, assemble, bring up, find the mistake, respin. Two are drone boards, small enough to finish quickly and sensitive to power and layout errors. Two are server boards: Board A is the management controller that becomes Module Zero's BMC, and Board B is a PCIe host board on which you place every trace. Each turn of the cycle takes weeks even at a fast fabricator and months at a contract manufacturer, and that lead time shapes every schedule and cost model in Chapter 4.6.

SKIP TEST

1. A 3.3 V rail feeds a load drawing 400 mA with 50 mV of allowable ripple from a buck switching at 1 MHz. Size the output capacitance and its ESR, then say why a 100 nF ceramic still goes next to every power pin.
2. What is the characteristic impedance of a 0.2 mm trace over a 0.2 mm prepreg to the reference plane on a typical four-layer board, and how long can a 25 MHz SPI clock with 1 ns edges run before you treat it as a transmission line?
3. A PCIe Gen 3 lane pair leaves a connector, changes layer through a via and reaches an M.2 slot. List the layout rules you must meet and the two you can bend.
4. Your new board draws 1.2 A at power-on instead of 200 mA. Give the first five things you do, in order, before touching firmware.
5. What does a design-for-manufacture review check, and which three of your choices most affect the assembled cost at a hundred pieces?

If all five are easy, do Labs 3.4.5, 3.4.7 and the design review only.

Core ideas

The power tree. Start every design from the power tree you drew in Lab 1.4.5: input, protection, each rail with its current, sequencing and tolerance, and the regulator that produces it. A switching regulator is efficient but noisy, and a linear regulator is quiet but turns its dropout voltage times the load current into heat. Every rail needs bulk capacitance at the regulator and decoupling at each load pin. Multi-rail parts state a power-up order in their datasheets, and a sequencer or a chain of enable pins enforces it. Inrush,

brownout, reverse polarity and hot plug are the failure modes you protect against, and a current-limited bench supply protects the board at its first power-on.

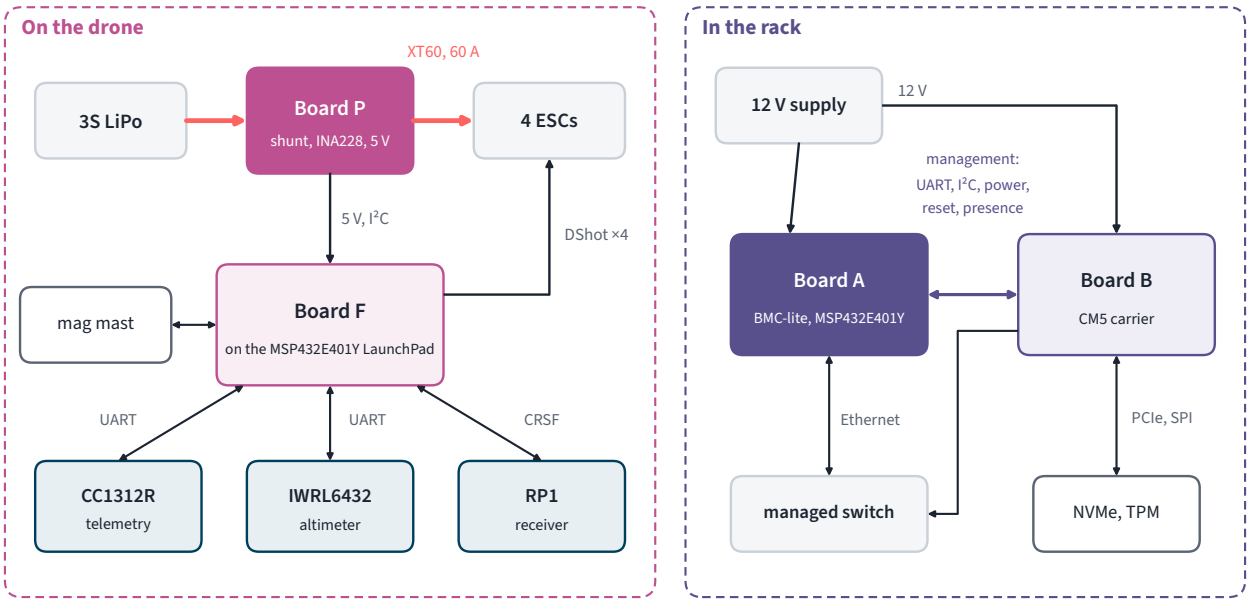


Figure 3.4.1. The four boards and their connections. P and F fly on the drone, and A and B become Module Zero's first node in Chapter 4.7.

The four boards.

- **Board P**, the drone's power module, on two layers with 2 oz copper: XT60 in and out on a 60 A path, a 0.5 mΩ four-terminal shunt read by an INA228, a TPS62933 buck making 5 V at 3 A for the electronics, reverse-polarity protection and a TVS on that low-current branch only, two USB-A sockets that supply 5 V to the radio and radar LaunchPads, and a JST-GH connector to Board F carrying 5 V, I²C and the INA228's alert.
- **Board F**, the flight-controller BoosterPack for the MSP-EXP432E401Y, on four layers: the LSM6DSOX and BMP581 as chips, a W25Q128JV SPI flash for the flight logger, four ESC outputs with series resistors and ESD protection, UART connectors for the receiver, the telemetry radio and the radar, an I²C connector to the magnetometer mast, a 3.3 V regulator, a buzzer, LEDs and test points. The LaunchPad takes its power from the BoosterPack header when its JP1 jumper is in the left position, and in that position the board must also be given 3.3 V with the J101 3V3 jumper removed (TI SLAU748B section 2.1.6.4); Board F's regulator provides it.
- **Board A**, the BMC-lite, on four layers: the MSP432E401Y in its 128-pin package with the crystal and Ethernet magnetics from the LaunchPad's reference design, a 12 V input through a TPS25947 eFuse to a 3.3 V buck, an INA228 on the input, two TMP117s at the board's edges, two 4-pin fan headers with PWM and tachometer, an ATECC608 for identity, an RJ45 MagJack, a 3-pin JST-SH SWD connector that takes the Debug Probe's cable, status LEDs, and a host connector carrying UART, I²C, power-good, reset, power-enable and presence. The outline is a 100 × 80 mm rectangle, or OCP's DC-SCM 2.0 card if you want the harder option.
- **Board B**, the Compute Module 5 carrier, on four or six layers with controlled impedance: the CM5's two connectors, an M.2 M-key slot for NVMe on the CM5's PCIe lane, an SPI TPM (Infineon SLB 9672), Gigabit Ethernet to a MagJack, 12 V input through an eFuse to the CM5's 5 V supply, USB for a keyboard, and the management connector to Board A.

Schematic capture as specification. Treat a schematic as a document first. Use one sheet per function, named nets, reference designators grouped by function, and a note wherever a value came from a calculation. Every symbol is checked pin by pin against the datasheet's pin table, and every footprint against its mechanical drawing, with a 1:1 paper print of the layout before ordering. The electrical rules check catches unconnected pins and driver conflicts. Rules you can say in words (every enable has a defined default; every I²C bus has pull-ups and a test point) become the checklist at the end of this chapter.

Component selection and the supply chain. The part you want is often out of stock, as Units 1 to 3 have shown. Choose active parts with second sources and more than one package, check stock at two distributors, and prefer the assembler's basic library for anything you do not care about. A bill of materials with manufacturer and distributor part numbers, alternates and price breaks is a deliverable. Connectors and branded parts dominate the cost, and passives add little to it.

Stack-up and impedance. A four-layer board (signal, ground, power, signal) with thin prepreg between each outer layer and its plane gives controlled-impedance traces and a solid return path. Width and height above the plane set impedance; your fabricator publishes its stack-up and a calculator. Targets: 50 Ω single-ended, 90 Ω differential for USB and PCIe, 100 Ω for Ethernet. Use six layers when you need more than two signal layers, each with an adjacent plane.

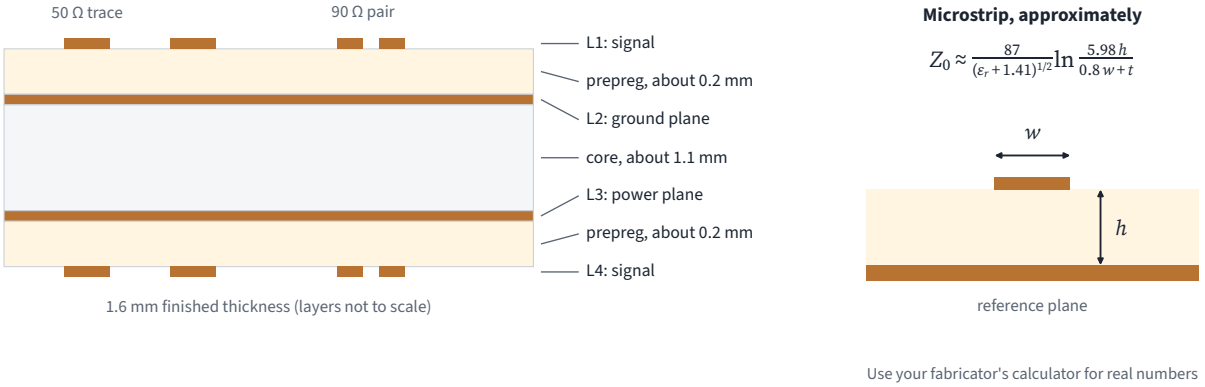


Figure 3.4.2. A four-layer stack-up and the microstrip that sits on it. Impedance depends on the trace width w against its height h above the plane, which is why a stack-up is chosen before the layout starts.

Signal integrity. A trace is a transmission line when it is longer than about a sixth of the distance an edge travels during its rise time, which for 1 ns edges is a few centimetres whatever the clock rate (Chapter 1.5). Terminate, match impedance, avoid stubs, keep the return path under the trace, and change reference plane only with a stitching via or capacitor nearby. Differential pairs need matched lengths within a fraction of the bit period (about 0.13 mm for PCIe Gen 3), constant spacing, symmetric vias and no plane split beneath them. A source-synchronous bus's timing budget is Chapter 2.1's setup-and-hold analysis with trace delay (about 6 ps per millimetre) added.

Power integrity and EMC. The power distribution network is an impedance against frequency; its target is the allowed ripple divided by the transient current. Bulk capacitors cover kilohertz, ceramics megahertz, and the plane pair above that. Every current loop radiates, so keep loops small and returns adjacent, filter what leaves the board, and give the enclosure a ground. The loops that matter are the 60 A path on Board P, the buck's switching loop on Board A, and the PCIe and Ethernet pairs on Board B.

Layout. Place power first, then the critical signals (clocks, high-speed pairs, the IMU and the shunt's Kelvin traces), then the rest. Route the critical nets by hand. Pour ground with stitching vias. Use thermal reliefs on pads you solder by hand and solid connections where current flows. Add test points on every rail, reset, enable and bus, fiducials for assembly, silkscreen that names every connector and marks pin 1, and mounting holes with keep-outs. Run a design-rule check against your fabricator's capabilities before export.

Fabrication and assembly. The order needs Gerbers, drill files, a pick-and-place file and a BOM in the assembler's format. Design for manufacture means consistent orientation, nothing under other parts, no 0201 unless you must, and tented vias under the stencil. Design for test means probe points a fixture can reach and a way to program and reset the board without hands. Controlled impedance and a custom stack-up cost extra; order them for Board B.

Bring-up. Write the bring-up procedure before the boards arrive. Inspect under the microscope for bridges, tombstones and missing parts; check every rail to ground with nothing powered; power on with the bench supply limited to a fraction of the expected current, one rail at a time if you can; check each

rail's voltage and ripple, then the clock, then reset; attach the debugger and read the chip's ID; only then run code. Every failure gets a root cause, a fix, and an entry in the errata the respin must address.

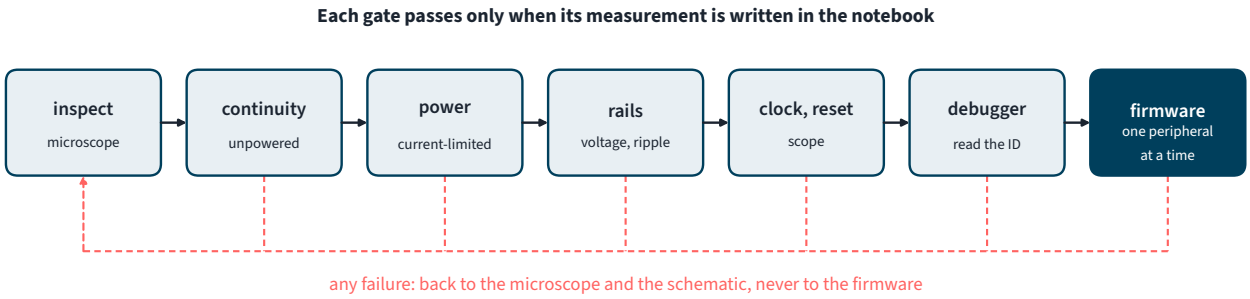


Figure 3.4.3. The bring-up procedure as a sequence of gates. You do not pass a gate until its measurement is in the notebook, and a failure at any gate sends you back to the microscope before you look at firmware.

Mechanical and thermal. Board F's IMU sits near the frame's centre of rotation, on foam, away from the ESC wiring. Board P's shunt and XT60 pads dissipate watts; a thermal camera or thermocouple shows whether the copper is enough. Board A lives in an enclosure with the fan curve you control; Board B's outline is set by the CM5, the M.2 drive and the connectors. A 3D export from KiCad catches collisions for a tenth of the cost of a respin.

Standards to borrow. OCP's DC-SCM 2.0 defines a management card that any host board accepts; designing Board A in its shape lets Module Zero later use a commercial card or offer its own. The M.2 and PCIe CEM specifications define a slot that standard drives and cards fit. Raspberry Pi publishes the CM5 IO Board's KiCad files; Board B starts from them.

Reading

- Horowitz and Hill, *The Art of Electronics*, 3rd ed., chapter 9 (regulation and power conversion), chapter 12 (logic interfacing) and appendix H (transmission lines).
- Bogatin, *Signal and Power Integrity, Simplified*, 3rd ed., chapters 1 to 8, then chapter 13 (the PDN) before Board A's power design.
- Johnson and Graham, *High-Speed Digital Design*, chapters 1 to 6, 8 and 12.
- Ott, *Electromagnetic Compatibility Engineering*, chapters 3, 10, 11 and 12.
- TI SLAU748B and the MSP-EXP432E401Y design files: schematic, layout and BOM, the reference for Board A's MCU, crystal and Ethernet; TI's application note on the MSP432E4 Ethernet PHY layout.
- The datasheets of every regulator you place, each with its layout guidelines section; the TPS62933, TPS25947 and INA228 datasheets at minimum.
- OCP DC-SCM 2.0 (management card form factor and connector); the PCI Express CEM specification chapter 4 and the M.2 specification's PCIe section, for Board B.
- Raspberry Pi's *Compute Module 5 Datasheet* and the CM5 IO Board design files.
- Your fabricator's capabilities page, stack-up table and DFM guide, before layout.

Labs

The boards go to fabrication in two batches so the lead times overlap with work: P and F together, then A and B. The design review checklist at the end of the chapter is mandatory for each board before ordering.

LAB 3.4.1 Specify four boards — 6 h

Goal. decide what each board is for and how you will know it works, before drawing anything.

1. Write one page per board: purpose; interfaces with pinouts; the power budget per rail; outline and mounting; the main parts and why; the bring-up plan; and what "working" means as measurable criteria.
2. For Board F, map every signal to an MSP432E401Y pin using the LaunchPad user's guide's BoosterPack tables, with no pin used twice: SSI for the IMU and the flash, four timer capture/compare pins for the ESCs, three UARTs, two I^2C buses, an ADC input for the battery divider, and GPIOs for the buzzer and LEDs.
3. For Board A, map the same functions plus Ethernet, the fans and the host connector, and confirm in the MSP432E401Y datasheet that each function exists on the pin you chose in the 128-pin package.

DONE WHEN

- [ ] Four specifications exist, each with a pin map checked against the datasheet, and every main part is in stock at two distributors.

EVIDENCE FOR THE NOTEBOOK

- The four pages; the pin maps; stock screenshots.

LAB 3.4.2 Board P: the drone's power module

10 h

Goal. design a board where layout decides whether it works.

Kit. KiCad, your Lab 1.4.5 drone power tree, the Lab 1.3.6 error budget.

1. Schematic: the 60 A path with the shunt in the positive lead, sized from your error budget; the INA228 with its Kelvin sense traces and address straps; the TPS62933 at 5 V and 3 A designed from its datasheet (inductor, capacitors, feedback divider, soft start); reverse-polarity protection and a TVS on the 5 V branch only; the USB-A sockets (power pins only); the connector to Board F.
2. Simulate the buck's output filter in ngspice for ripple under a 1 A load step.
3. Layout on two layers with 2 oz copper: the high-current path as wide pours with the XT60 pads sized for the wire, the shunt's sense traces leaving its sense pads as a pair, and the buck's input loop as small as the datasheet's layout guide shows.
4. Estimate the copper's temperature rise at 60 A for 10 s with an IPC-2152 calculator and widen what needs it.

DONE WHEN

- [ ] The board passes the design review checklist with no open findings and is ordered with Board F.

EVIDENCE FOR THE NOTEBOOK

- Schematic and layout PDFs; the buck design arithmetic and simulation; the temperature-rise estimate; the review record.

LAB 3.4.3 Board F: the flight-controller BoosterPack

2 weekends

Goal. put the drone's sensors and connectors on a board you designed.

Kit. KiCad, the Lab 3.4.1 pin map, the LaunchPad's BoosterPack mechanical drawing.

1. Schematic: the LSM6DSOX and BMP581 with decoupling as their datasheets show; the W25Q128JV; ESC outputs through 100 Ω series resistors and a TI ESD array; the UART, I^2C and power connectors (JST-GH); a 3.3 V regulator sized for the LaunchPad's MCU domain plus your sensors; a buzzer on a MOSFET; LEDs; test points on every bus.

2. Layout on four layers in the BoosterPack outline, with the headers on the LaunchPad's 0.1 inch grid. Place the IMU at the board's centre, away from the ESC traces and the regulator, with its axes printed on the silkscreen.
3. Print the layout 1:1, push the LaunchPad's headers through the paper, and check alignment.
4. Order P and F from a fabricator with assembly of the fine-pitch parts; hand-solder the connectors yourself.

DONE WHEN

- [ ] The board passes the checklist, the paper fit check is photographed, and P and F are ordered.

EVIDENCE FOR THE NOTEBOOK

- Schematic and layout; the fit-check photograph; the BOM with cost per board; the order confirmation with lead time and cost, which feed Chapter 4.6's cost model.

LAB 3.4.4 Board A: a BMC-lite on the MSP432E401Y

3 weekends

Goal. design the management controller Module Zero will run on.

Kit. KiCad, the LaunchPad's design files, your Chapter 2.6 firmware.

1. Start from TI's LaunchPad schematic for the MCU, its crystals, the Ethernet PHY's connections and the reset and debug circuitry. Understand every net you keep and cite the reason in a note.
2. Add the 12 V input with the TPS25947 eFuse and a 3.3 V buck, the INA228 on the input, the TMP117s, the two fan headers with tachometer pull-ups and PWM at 25 kHz, the ATECC608, the host connector, the debug header and LEDs. Put ESD protection on every connector that leaves the board.
3. Simulate the buck's output in ngspice for ripple under a 500 mA step, and design the 3.3 V PDN to a target impedance (problem 1).
4. Layout on four layers: Ethernet pairs at 100 Ω differential from the PHY pins to the MagJack with no vias, the crystal close and guarded, the buck's switching loop tight, SPI and I^2C routed over solid ground.
5. Review, including one outside review (a colleague, or a forum such as r/PrintedCircuitBoard), and order five assembled.

DONE WHEN

- [ ] The review record has no open findings, including those from the outside reviewer, and the order is placed.

EVIDENCE FOR THE NOTEBOOK

- Schematic with notes; layout; the PDN design; the review record; the order with cost and lead time.

LAB 3.4.5 Board B: a Compute Module 5 carrier

3 weekends

Goal. route PCIe and Gigabit Ethernet on a carrier board you design.

Kit. KiCad, the CM5 IO Board design files, the CM5 datasheet.

1. Strip the CM5 IO Board design to what you need. Add the M.2 M-key slot, the SPI TPM, the management connector to Board A, and the 12 V input with an eFuse and the 5 V supply the CM5 datasheet specifies.
2. Route PCIe: 90 Ω differential, matched within 0.13 mm in each pair, AC-coupling capacitors where the specification puts them, the reference clock pair with the same care, `PERST#` and `CLKREQ#` defined. Keep within the CM5 datasheet's length guidance.

3. Choose four or six layers so every high-speed pair has an adjacent plane. Order the fabricator's controlled-impedance stack-up and confirm your widths with its calculator.
4. Review with the high-speed additions to the checklist, and order five with the CM5 connectors and M.2 slot assembled.

DONE WHEN

- [ ] The order is placed, the fabricator's impedance report matches your targets, and the review has no open findings.

EVIDENCE FOR THE NOTEBOOK

- Schematic; layout; stack-up and impedance calculations; the review record.

LAB 3.4.6 Bring-up of Boards P and F

2 weekends

Goal. turn two bare boards into the drone's power and sensor core.

Kit. Both boards, the bench supply, electronic load, scope, microscope, Debug Probe, a LaunchPad, the 3S pack, thermal camera or thermocouple.

1. Inspect every joint under the microscope and photograph anything doubtful. Check each rail to ground and each connector pin to its net with nothing powered.
2. Board P on the bench supply at 12 V limited to 100 mA with no load: check 5 V, then raise the limit to 2 A and load the 5 V rail to 3 A with the electronic load and measure efficiency, ripple and a load step. Then 15 A through the main path from the pack into the electronic load for 60 s (the load's 200 W rating is the limit at this voltage): measure the shunt's and the pads' temperatures and the INA228's reading against the load's. The 60 A bursts are checked in Chapter 3.5's flight logs.
3. Board F on the LaunchPad with JP1 moved to the BoosterPack position and the 3V3 jumper on J101 removed, powered first from the bench supply through Board F's power connector. Read the IMU's and barometer's ID registers, the flash's JEDEC ID and the INA228 over the cable from Board P.
4. Port your Chapter 3.1 IMU and estimator code to Board F's pins and compare its noise with the breakout's. Write and read back the whole flash.
5. Record errata: every deviation, workaround (a cut trace, a wire, a swapped part) and what the respin changes.

DONE WHEN

- [ ] Board P delivers 5 V at 3 A within its ripple specification and carries 15 A for 60 s with a temperature rise within your Lab 3.4.2 estimate scaled to that current and time, with the INA228 within 1 % of the load.
- [ ] Board F's sensors and flash work on the LaunchPad from Board P's power, and its IMU noise is no worse than the breakout's.

EVIDENCE FOR THE NOTEBOOK

- Bring-up notebook with photographs, captures and thermal readings; the errata lists.

LAB 3.4.7 Bring-up of Boards A and B

3 weekends

Goal. make your management controller run your BMC firmware and control your host board.

Kit. Both boards, a 12 V supply, the Debug Probe, the CM5, an NVMe drive, the managed switch, the server.

1. Board A: inspection and continuity, then 12 V limited to 100 mA. Check the eFuse's inrush and the 3.3 V rail. Connect the Debug Probe, read the MCU's ID with OpenOCD and flash a blink.
2. Port the Chapter 2.6 firmware set: UART console, sensors, fans with tachometers, the INA228, the watchdog and the signed A/B bootloader. Bring up Ethernet with a link and a ping.
3. Board B without the CM5: rails and inrush on the scope. With the CM5: boot from eMMC with a serial console on the management connector, then check the Ethernet link. `lspci` must show the

NVMe drive; if it does not, check `dmesg` for link training, then `PERST#` and the reference clock on the scope, then force Gen 1 to separate a signal-integrity problem from a logic one.
4. Bring up the TPM: `/dev/tpm0` with your Chapter 2.7 device-tree overlay, `tpm2_pcrread`, and the measured-boot chain from Lab 2.7.4 with your U-Boot measuring the kernel.
5. Connect A to B. Board A controls B's power and reset, reads its presence, forwards its console and runs a fan from B's temperature. Power-cycle B from A's console.
6. Boot B through your Lab 2.7.6 zero-touch path: netboot, install, attest, join.

DONE WHEN
- ☐ Board A runs the Chapter 2.6 firmware set, updates itself over UART, and answers ping over its own Ethernet.
- ☐ Board B boots Linux from NVMe through your U-Boot with the TPM measuring the chain, under Board A's power control, and your attestation verifier admits it.

EVIDENCE FOR THE NOTEBOOK
- Bring-up notebooks; captures; the errata lists.

LAB 3.4.8 The respin decision — 4 h

Goal. decide whether to respin each board from its errata, costs and lead times.

1. Consolidate the four errata lists. Classify each item: must fix (the board does not work without a workaround), should fix (works, but is not manufacturable or reliable), could fix.
2. For each board, decide what a respin changes and estimate the cost and lead time for five more.
3. Respin any board with a must-fix item that blocks Chapter 3.5 or 4.7. Write the decision as a page with the numbers behind it.

DONE WHEN
- ☐ The decision is written with costs, lead times and the reason for each choice.

EVIDENCE FOR THE NOTEBOOK
- The consolidated errata and the decision page.

The design review checklist

Run this on every board before ordering, and add to it after every bring-up.

DESIGN REVIEW

A board is not ordered until every line is ticked or has a written exception.

- ☐ **Power:** every rail's budget summed and within its regulator's rating with margin; input protection; sequencing meets every datasheet; bulk and per-pin decoupling; a test point per rail; a way to measure each rail's current.
- ☐ **Resets and enables:** defined at power-on by a resistor, never floating; a supervisor or RC where a datasheet requires it; reset reachable from the debugger.
- ☐ **Clocks:** crystal load capacitors computed from the crystal's specification; short traces; ground guard; nothing routed underneath.
- ☐ **Buses:** I^2C pull-ups sized for the bus capacitance; addresses unique on each bus; SPI chip selects pulled inactive; UART voltage levels match at both ends.
- ☐ **High-speed:** impedance, length match, via count, reference-plane continuity, AC coupling, keep-outs, and the source's maximum length.
- ☐ **Connectors:** pin 1 marked; mating part in the BOM; mechanical keep-out; ESD on everything external; current rating per contact.

- ☐ **Debug:** SWD or JTAG header; console UART; an LED on power-good and on a GPIO; test points on every net you might scope.
- ☐ **Mechanical:** outline, holes, standoff height, connector overhang, a 3D collision check, and a 1:1 paper print with real parts placed on it.
- ☐ **Manufacturing:** footprints checked against datasheets; consistent orientation; fiducials; stencil apertures; every BOM line with a manufacturer number, a distributor number and an alternate; parts in stock.
- ☐ **Documentation:** a note for every calculated value; net names; revision and date on the silkscreen; the bring-up procedure written before the boards ship.

Problem set

1. Design the PDN for Board A's 3.3 V rail: transient current, ripple budget, target impedance, and a capacitor set whose combined impedance stays below the target from 1 kHz to 100 MHz. Show the curve.
2. Board P's shunt is 0.5 mΩ. At 60 A, what power does it dissipate? If the copper between the shunt's current pads and its sense pads adds 50 μΩ that the Kelvin connection fails to exclude, what error does it cause?
3. The SPI bus from Board F's MCU to the flash is 40 mm long at 30 MHz with 1 ns edges. Is it a transmission line? Decide on termination and compute the setup margin with trace delay included.
4. Board B's PCIe pair changes from the top layer to the bottom through two vias. Explain what the return current does at each via, what you add to help it, and estimate the discontinuity a via adds in picoseconds.
5. Board A's 12 V input sees a 30 V transient for 100 μs when the bench supply is switched on. Choose a protection scheme and its parts.
6. Estimate the assembled cost of Board A at 5, 100 and 1,000 units, split into fabrication, assembly, connectors, silicon and passives. Which design decision most affects the 1,000-unit price?
7. Board A's first unit draws 800 mA at 12 V with nothing running. Give an ordered diagnostic procedure and the three most likely causes.
8. A part in your BOM goes end-of-life six months after your first production run. What in your design and process decides whether that is a one-week or a six-month problem?
9. Compare designing Board A in the DC-SCM 2.0 form factor with a custom outline, for Module Zero: what does the standard buy, what does it cost, and what would a customer with an existing fleet think?

Deliverables and stretch

Deliverables. Four specifications, four KiCad projects, four review records, four ordered BOMs with costs and lead times; bring-up notebooks with photographs, captures and errata; working Boards P and F feeding the drone; Board A running your BMC firmware; Board B booting your chain from NVMe under Board A's control; the respin decision.

Stretch. Respin Board A with an ECP5 FPGA and the Chapter 2.2 register-map peripheral on board. Design a sled for Boards A and B in the Open Rack v3 or Yosemite v3 envelope in FreeCAD and print a mock-up. Build a pogo-pin test fixture for Board A that programs the bootloader and runs a self-test, and time a unit through it: Chapter 4.6's manufacturing test in miniature. Scan Board A for radiated emissions with the Tekbox near-field probes and the tinySA, find the loudest loop and fix it.

COMPLETION CHECKLIST

- ☐ **Lab 3.4.1:** four specifications with checked pin maps; parts in stock at two distributors.
- ☐ **Lab 3.4.2:** Board P reviewed and ordered, with the buck design and the temperature estimate.
- ☐ **Lab 3.4.3:** Board F reviewed, fit-checked on paper and ordered.

- ☐ **Lab 3.4.4:** Board A reviewed with an outside reviewer and ordered.
- ☐ **Lab 3.4.5:** Board B ordered with a matching impedance report.
- ☐ **Lab 3.4.6:** Board P at 3 A and 15 A within specification; Board F's sensors and flash working on the LaunchPad.
- ☐ **Lab 3.4.7:** Board A running your BMC firmware over its own Ethernet; Board B booting the measured chain from NVMe under A's control.
- ☐ **Lab 3.4.8:** the respin decision written with numbers.
- ☐ **Problem set:** all nine answered.
- ☐ **You can explain**, without notes: why a board starts from its power tree, what sets a trace's impedance, how a return current behaves at a via, what you check before applying power to a new board, and what decides whether to respin.

CHAPTER 3.5

The drone: integration, test and flight

TIME	5 weeks
YOU BUILD	A 450 mm quadcopter that flies on your firmware: DShot motor output over µDMA, a CRSF receiver parser with failsafe, a flight-software core with an arming state machine, a mixer, a flight logger and telemetry, tested in software-in-the-loop simulation; an assembled airframe with your Boards P and F, the 868 MHz telemetry radio and the 60 GHz radar altimeter; rig-tuned rate and angle loops; tethered then free hover; altitude hold fusing radar and barometer; a written flight-test report.
NEEDS	Everything from Chapters 3.1 to 3.4: the Tarot frame, four SunnySky motors and CloudPhoenix ESCs, props, the 3S pack and charger, Boards P and F on an MSP-EXP432E401Y, the RP1 receiver and Pocket transmitter, the BMP581, the LIS3MDL on a mast, a CC1312R LaunchPad on the drone and one on the ground with the Pi, the IWRL6432BOOST, the one-axis rig, a tether line and a 10 kg anchor, safety glasses, a logic analyser.

WHAT THIS BUYS OMH

In integration the course's separate skills have to work together under a deadline, and mistakes have physical consequences. A server module and a drone fail in the same way, when subsystems that each passed their tests interact through timing, power, vibration, electromagnetic interference and software states nobody drew. The habits this chapter requires (a written system architecture, interface contracts, a simulator before hardware, a test plan with exit criteria, logs good enough to explain every anomaly, and a report) are what make Module Zero's integration in Chapter 4.7 possible, and what a customer's acceptance test of an OMH rack will demand.

SKIP TEST

1. Encode DShot600 throttle 1,046 with no telemetry request: give the 16 bits, including the checksum, and the frame's duration.
2. A CRSF channels frame packs sixteen 11-bit channels into 22 bytes. Write the expression that extracts channel 3 from the payload bytes.
3. List the conditions your firmware must check before it arms, and the events that must disarm it, with a time limit for each.
4. The radar reports 2.10 m to the ground with the drone pitched 12°. What is the height, and what error does a 2° attitude error add?
5. Your rate loop runs at 1 kHz. Budget the time for the IMU read, estimator, controllers, mixer and DShot set-up, and say what happens if the budget overruns once a second.

If all five are easy, do Labs 3.5.3, 3.5.6 and 3.5.7.

Core ideas

The system. The flight controller is an MSP-EXP432E401Y carrying Board F; Board P supplies it and measures the battery. Four ESCs take DShot from four timer outputs. The RP1 receiver sends CRSF over a UART. The CC1312R LaunchPad on the drone forwards telemetry frames from another UART to the ground. The IWRL6432BOOST, facing down, reports range over a third. The magnetometer sits on a mast at the height Lab 3.1.6 chose. Write this as an architecture document before you assemble anything: every interface with its protocol, rate, voltage level and owner.

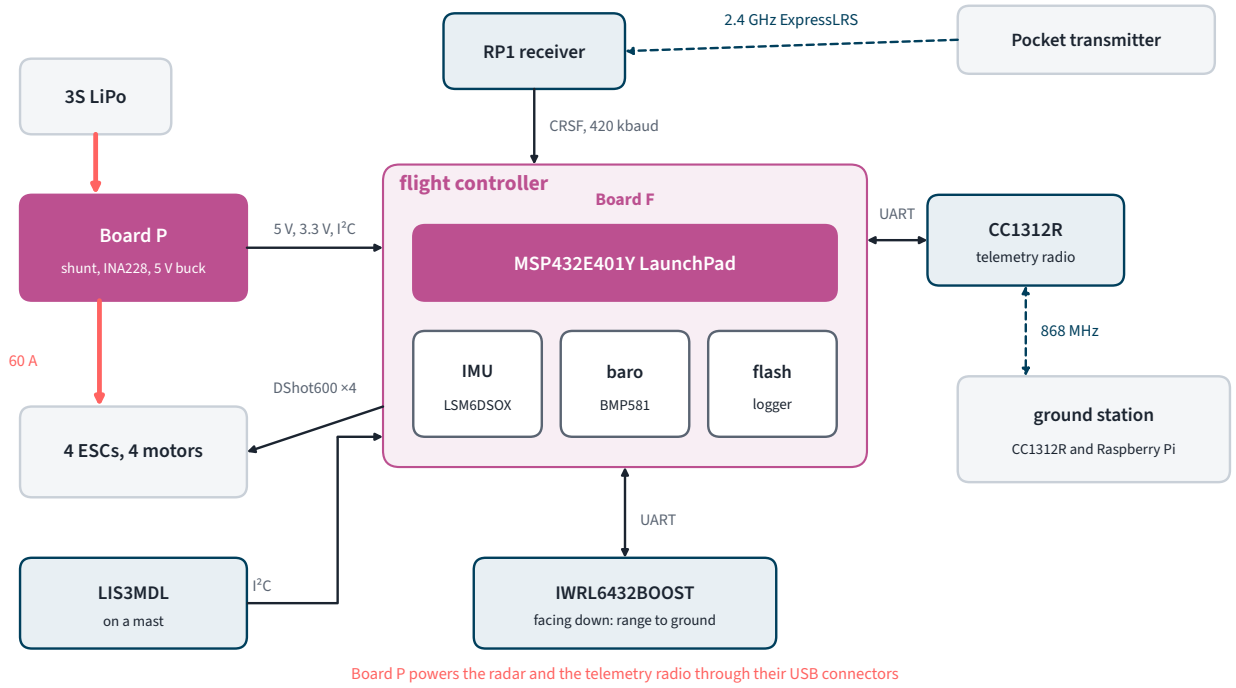


Figure 3.5.1. The drone as a system. Every arrow is an interface with a protocol, a rate and a failure mode, and each one appears in the architecture document you write in Lab 3.5.3.

DShot. DShot is a digital ESC protocol. Each frame is 16 bits: an 11-bit throttle (0 disarmed, 1 to 47 commands, 48 to 2,047 throttle), one telemetry-request bit, and a 4-bit checksum, the XOR of the three nibbles of the first 12 bits. Each bit is a pulse in a fixed period, high for 75 % of it for a one and 37.5 % for a zero. At DShot600 the bit period is 1.67 µs and a frame takes 26.7 µs. Because it is digital it needs no calibration, has a checksum, and carries commands such as spin direction and beeps. Generating it in software would cost the CPU every microsecond; a timer in PWM mode whose compare value µDMA reloads once per bit costs nothing per bit. Bidirectional DShot inverts the line and lets the ESC answer with motor speed, which flight controllers use to place their notch filters.

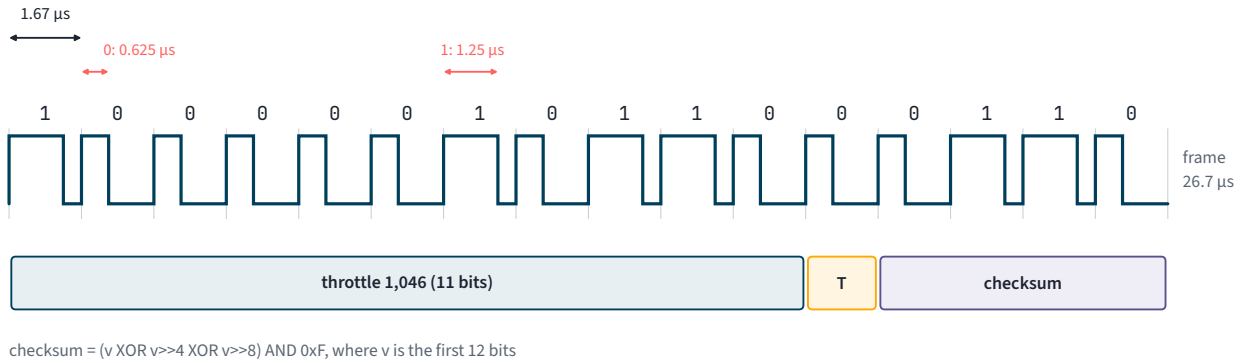


Figure 3.5.2. One DShot600 frame. The timer's period is the bit time; µDMA writes the next compare value at each period's start, so the CPU's only work per frame is writing sixteen words.

The control link. ExpressLRS sends stick positions over 2.4 GHz at a packet rate you choose (50 to 1,000 Hz), and the RP1 delivers them as CRSF at 420,000 baud: a frame of an address byte, a length, a type, a payload and a CRC-8 with polynomial 0xD5 over the type and payload. The channels frame carries sixteen 11-bit channels, with 172 to 1,811 representing 988 to 2,012 µs. Link statistics frames report signal strength and link quality. When the link fails the receiver stops sending channel frames and sends no failsafe message, so failsafe is a timeout in your code.

Arming and failsafe. Most drone accidents happen on the ground, when a motor spins while someone is holding the frame. An arming state machine makes the dangerous transitions explicit. It refuses to arm unless the arm switch moves from off to on with throttle low, the attitude is level, the IMU is calibrated,

the battery is above its threshold and the link is healthy. It disarms on the switch, on loss of link beyond a timeout, on an attitude beyond any flyable angle, and on a critical battery. Each transition is logged with its reason. In this course a lost link disarms the drone. Because the drone never flies higher than a fall it can survive, a controlled descent is left as a stretch goal.

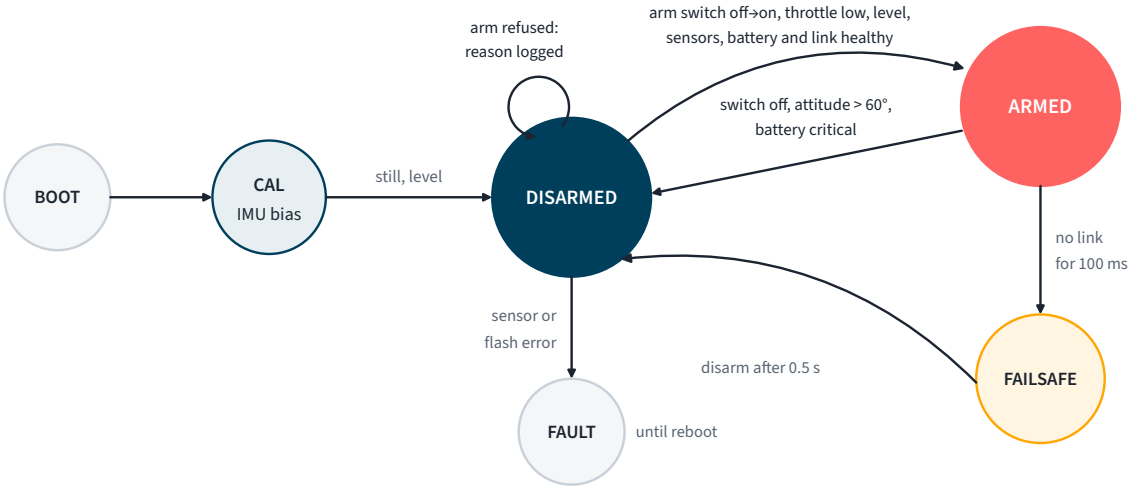


Figure 3.5.3. The arming state machine. Every arrow is a test case in Lab 3.5.3's simulator, and the guarded transitions into ARMED are the ones that prevent injuries.

The control core. The IMU's data-ready interrupt at 1.66 kHz starts the fast path: read the IMU by µDMA, update the attitude estimate, run the rate controllers, mix, and start the DShot transfer. The angle loop runs at 500 Hz, altitude at 50 Hz, telemetry at 10 Hz and the logger at 500 Hz from lower-priority contexts. The mixer turns collective thrust and roll, pitch and yaw demands into four motor commands; when one saturates, it must give up altitude to keep attitude control.

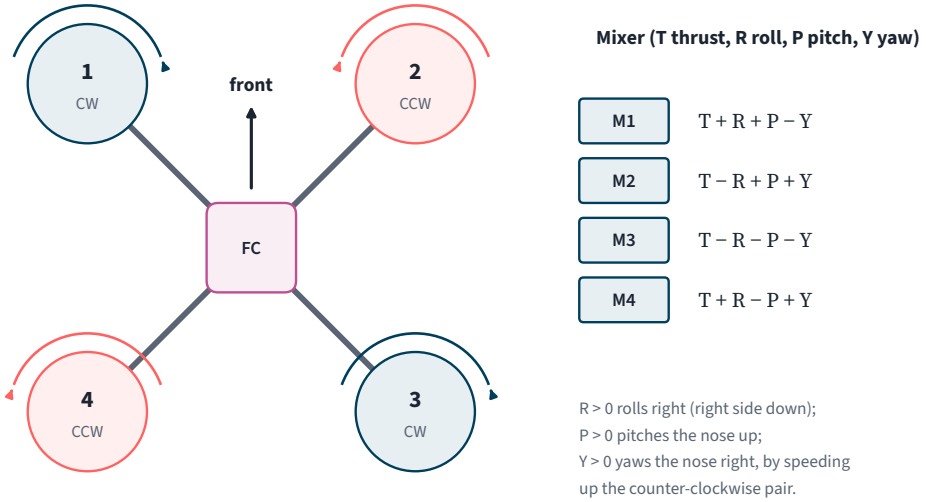


Figure 3.5.4. Motor numbering, rotation directions and the mixer for the course's X configuration. Diagonal motors spin the same way, and the two diagonals spin in opposite directions so their reaction torques cancel. Yaw comes from speeding up one diagonal and slowing the other.

Simulation before flight. The same control code, compiled for your workstation, runs against a six-degree-of-freedom rigid-body model with the motor lag and thrust curve from Lab 3.1.1, sensor noise and bias from Labs 3.1.4 and 3.1.6, and the loop delays you measured. Software-in-the-loop testing finds sign errors, mixer mistakes, arming bugs and saturation behaviour at no risk. It does not prove that the drone will fly, but each bug it finds is fixed before any propeller is fitted.

Altitude. Three sensors measure height, each with a different weakness. The barometer reads pressure altitude with about 10 cm of noise, drifts with weather and temperature, and sees prop wash. The radar

measures range to the ground to centimetres, but range is height only when level ($h = r\cos\theta\cos\varphi$ over flat ground), and it drops out over water and at steep angles. The accelerometer, rotated to the earth frame with gravity removed, gives vertical acceleration with no delay but drifts within seconds when integrated. A three-state Kalman filter (height, vertical velocity and barometer bias) fuses them: the accelerometer drives the prediction, the radar and barometer correct it, and the bias state lets the barometer track the radar while the radar is valid and coast when it is not.

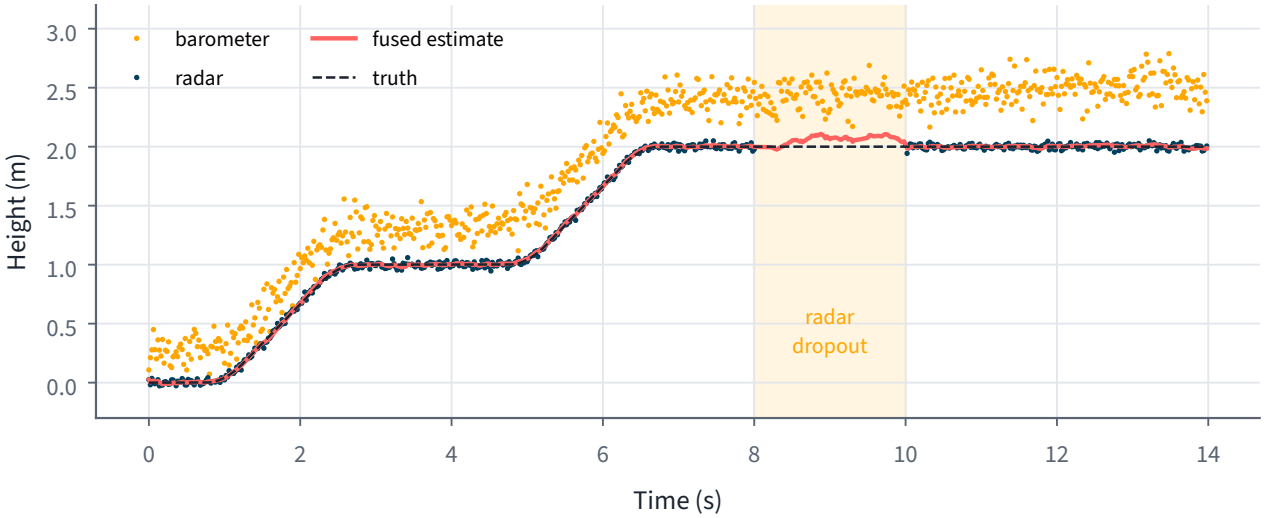


Figure 3.5.5. Altitude fusion on a simulated hover with a radar dropout from 6 to 8 s. The fused estimate follows the radar while it is valid and coasts on the barometer, with its learned bias, during the dropout.

Testing like an aircraft. Each test has a card: the objective, the configuration, the procedure, the abort criteria and the data to collect. Tests climb a ladder from software to hardware: simulation, props-off bench, the one-axis rig, a tether, free hover at low height, then the manoeuvre. You climb one rung at a time and step down when an anomaly is unexplained. The flight logger records enough to explain any anomaly afterwards: sensor data, estimates, setpoints, controller terms, motor outputs, battery and link state, and every state transition.

Reading

- Your own Chapter 3.1 and 3.2 deliverables, and the TI SLAU723A chapters on general-purpose timers (PWM mode and the GPTMDMAEV register) and µDMA.
- The DShot and bidirectional DShot descriptions in the Betaflight wiki and source (`src/main/drivers/dshot.c`), read after you have written your own.
- The TBS CRSF protocol specification, and the ExpressLRS documentation on packet rates, telemetry ratio and failsafe.
- Beard and McLain, *Small Unmanned Aircraft*, chapters 3 to 5 (the equations of motion you simulate); or Mahony, Kumar and Corke, "Multirotor Aerial Vehicles" (IEEE Robotics and Automation Magazine, 2012).
- Simon, *Optimal State Estimation*, chapter 5 (the discrete Kalman filter), for the altitude filter.
- The PX4 or ArduPilot documentation on pre-flight checks, arming checks and log analysis, as examples of how mature projects structure them.

Labs

SAFETY

Propellers come off for every bench test, every firmware flash and every wiring change; they go on only for a rig, tether or flight test with a written test card. Wear safety glasses and keep everyone else at least 5 m away during powered tests. Test the kill switch before every flight. Fly over grass in

an open space with no one underneath. Charge, store and retire the pack by the Chapter 1.4 rules, and land at 3.5 V per cell.

LAB 3.5.1 DShot over µDMA — 8 h

Goal. drive four ESCs with a protocol you implemented, at zero CPU cost per bit.

Kit. Flight-controller LaunchPad with Board F, logic analyser, one ESC and motor without a propeller, the 3S pack.

1. Choose four timer capture/compare pins from your Board F pin map. Configure each half-timer in PWM mode with a period of 200 cycles at 120 MHz (1.67 µs). Establish from the TRM which timer event can request µDMA once per period in PWM mode, set it in GPTMDMAEV, and confirm on the logic analyser with a GPIO toggled in the transfer-complete interrupt.
2. Encode frames with the checksum, fill a buffer of compare values, and let µDMA send each frame. Add a reset gap between frames and send all four motors' frames in parallel.
3. Write a decoder for the logic analyser's exported samples that checks timing and checksum for every frame, and run it on 10,000 frames.
4. With the ESC on the pack and the motor bare: arm with zero throttle, send the beep and spin-direction commands as their specification requires, and ramp throttle from 48 to 300 and back.
5. Measure the CPU time per update and the latency from your throttle write to the first edge.

DONE WHEN

- [ ] Ten thousand frames decode with correct timing and checksum, and the motor arms, beeps, reverses and ramps under your commands.
- [ ] CPU time per update and the write-to-edge latency are measured.

EVIDENCE FOR THE NOTEBOOK

- Source; the decoder; captures; the timing measurements.

LAB 3.5.2 The receiver link — 6 h

Goal. turn stick movements into validated setpoints, and detect quickly when they stop.

Kit. RP1 receiver, Pocket transmitter, flight-controller LaunchPad, logic analyser.

1. Update the receiver to the transmitter's ExpressLRS major version and bind it. Connect it to the UART on Board F for the receiver at 420,000 baud.
2. Write the parser as a byte-at-a-time state machine fed from the UART's µDMA ring buffer: sync, length check, CRC-8 over type and payload, channel unpacking, link statistics. Count every kind of error.
3. Test it on your workstation against recorded byte streams with injected corruption: truncated frames, wrong lengths, bad CRCs, noise between frames.
4. Implement failsafe: no valid channels frame for 100 ms is a lost link. Switch the transmitter off and measure the time to detection on the analyser.
5. Measure the time from a channels frame's last byte to the change in DShot output.

DONE WHEN

- [ ] The parser passes every corruption test, and loss of link is detected within 120 ms on ten trials.
- [ ] Frame-to-output latency is measured and stated against the packet rate.

EVIDENCE FOR THE NOTEBOOK

- Parser source and tests; captures of failsafe detection and latency.

LAB 3.5.3 Flight software and simulation — 2 weekends

Goal. build the flight core and break it in simulation before it can break anything else.

Kit. Workstation, the flight-controller LaunchPad with Board F.

1. Write the architecture document: interfaces, rates, priorities, the timing budget of the 1.66 kHz path, and the failure response for every sensor and link.
2. Implement the core: the interrupt-driven fast path; the arming state machine with every guard and disarm reason logged; the mixer with saturation handling that preserves attitude; parameters in EEPROM with a version and CRC; the flight logger writing 500 Hz records to Board F's flash; telemetry frames to the radio bridge from Lab 3.2.4.
3. Build the same code for the workstation against a six-degree-of-freedom simulator with your measured motor model, sensor noise and delays. Write test cases for every arming guard, every disarm path, link loss, a stuck IMU, motor saturation during a roll, and a battery sag.
4. Fly simulated hovers and steps. Compare the simulated roll-rate loop with your Lab 3.1.7 rig data.

DONE WHEN

- [ ] Every test case passes in simulation, and the fast path's measured execution time on the MCU is under 50 % of its period.
- [ ] The logger records a full simulated flight and a script replays it into plots.

EVIDENCE FOR THE NOTEBOOK

- The architecture document; source; the test report; replay plots.

LAB 3.5.4 Assembly — 8 h

Goal. build an airframe that is balanced, low in vibration and wired so you can debug it.

Kit. Frame, motors, ESCs, Boards P and F with the LaunchPad, receiver, both radios, radar, magnetometer mast, standoffs, foam tape, cable ties, 14 AWG wire, XT60 connectors, heat-shrink, the scale from the thrust stand.

1. Mount motors and ESCs, solder the ESC power leads to the frame's power board and Board P between battery and frame. Mount the flight controller on foam at the frame's centre, the receiver antennas at 90° to each other away from carbon and wiring, the telemetry radio where Lab 3.2.5 put it, the radar facing down with nothing in its field of view, and the magnetometer on its mast.
2. For the radar and the telemetry radio, read each board's schematic to see how its application UART reaches the debugger and the BoosterPack headers. On the CC1312R LaunchPad, remove the RXD and TXD jumpers so the flight controller owns the UART. On the IWRL6432BOOST, set the switches the user's guide gives for routing the UART to the headers; if the debugger's transmit line cannot be isolated, decide how the radar gets its configuration and write the decision down.
3. With propellers off: check every motor's number and direction against the mixer figure, correct directions with DShot commands, and check the radio, radar and telemetry from the bench.
4. Weigh the drone and balance it: the centre of gravity within 5 mm of the frame's centre in both axes, adjusted by moving the pack.
5. Tethered to the rig with propellers on, run the motors to hover throttle and log the IMU at full rate. Compare the vibration spectrum with Lab 3.1.5's and fix anything new (a loose prop, an unbalanced motor, a stiff mount).

DONE WHEN

- [ ] Motor order, direction, centre of gravity and every sensor are verified with propellers off, and the vibration spectrum at hover throttle is no worse than on the thrust stand.

EVIDENCE FOR THE NOTEBOOK

- Photographs; the wiring diagram; the weight and balance record; the vibration spectra.

LAB 3.5.5 Rig tuning — 6 h

Goal. tune the airframe's loops on the rig, where a mistake does no damage.

Kit. The drone, the one-axis rig from Lab 3.1.7 adapted to the full frame, safety glasses, the transmitter.

1. Mount the drone on the rig for roll. Bring up the rate loop with low gains, then raise them while logging step responses from the transmitter's stick. Stop at the first sign of oscillation and back off by a third.
2. Close the angle loop and log steps and a hand-push disturbance. Compare with the simulation and update the model where it was wrong.
3. Remount for pitch and repeat. If you bought the three-axis rig, tune yaw on it; otherwise tune yaw in the first hover with conservative gains.

DONE WHEN

- ☐ Roll and pitch angle steps of 15° settle within 0.5 s with under 20 % overshoot on the rig, and the logged responses match the updated simulation in rise time within 30 %.

EVIDENCE FOR THE NOTEBOOK

- The logged step responses with the simulation overlaid; the final gains and the reasoning for them.

LAB 3.5.6 First hover — 1 day outdoors

Goal. fly a tethered and then a free hover, following written test cards.

Kit. The drone, a fully charged pack, the transmitter, a 1.5 m tether line tied to a 10 kg anchor, safety glasses, the test cards, a phone camera on a tripod.

1. Write the test cards: tethered hover, free hover at 1 m, gentle translation. Each has its objective, procedure, abort criteria and data to collect.
2. Pre-flight: propellers tight and correctly oriented, pack voltage per cell, centre of gravity, arming refused with throttle up, kill switch disarms, and failsafe disarms within 0.5 s of switching the transmitter off (propellers off for these last checks).
3. Tethered hover: tie the anchor line to the frame's centre so the drone cannot rise above 1.5 m or travel more than 1.5 m. Hover for 60 s in angle mode with manual throttle. Land and read the log before anything else.
4. Free hover at 1 m for 60 s, then gentle translations in each direction. Land at 3.5 V per cell.
5. From the logs, plot attitude error, motor outputs, current and vibration, and explain any anomaly before the next flight.

DONE WHEN

- ☐ A 60 s free hover holds attitude within ±5° with no motor saturating for more than 100 ms, recorded on video and in the log.
- ☐ Every anomaly in the logs is explained in the notebook.

EVIDENCE FOR THE NOTEBOOK

- The test cards with results; the video; log plots and the anomaly notes.

LAB 3.5.7 Altitude hold with radar and barometer — 8 h plus a day outdoors

Goal. make the drone hold a height by itself, from sensors you characterised.

Kit. The drone with the radar and barometer, tape measure, a pole marked at 1, 2 and 3 m.

1. Port your Lab 3.3.5 parser to the flight controller and run the radar at your altimeter configuration. Read the BMP581 at 50 Hz under a foam cover.

2. Implement the three-state Kalman filter with tilt compensation of the radar range and gating of radar outliers. Hold the drone by hand at measured heights from 0.3 to 3 m, tilted and level, and compare the estimate with the tape.
3. Implement altitude hold: a height loop commanding vertical velocity, a velocity loop commanding collective thrust with hover-throttle feedforward from your thrust model and battery voltage.
4. Fly holds at 1, 2 and 3 m beside the pole, steps between them, and passes over grass and concrete. Cover the radar for 2 s in flight to force a dropout.

DONE WHEN

- ☐ Hand-held estimates are within 5 cm of the tape when level and within 10 cm at 15° of tilt.
- ☐ In flight the drone holds each height within ±15 cm for 30 s and survives the forced dropout with under 30 cm of excursion.

EVIDENCE FOR THE NOTEBOOK

- Filter design and tuning; hand-held comparison table; flight logs with height estimates, setpoints and the dropout.

LAB 3.5.8 Telemetry and the flight report — 1 day plus writing

Goal. operate the drone from the ground and report on it like an engineer.

Kit. The drone, the ground station (CC1312R LaunchPad and the Pi), a fully charged pack.

1. Fly a scripted mission with altitude hold: climb to 2 m, hold 60 s, step to 3 m, hold 60 s, descend and land, with live telemetry on the Pi showing attitude, height, battery and link quality.
2. Fly a hover to the 3.5 V per cell landing threshold and record flight time and energy used. Compare hover power and flight time with your Lab 3.1.1 and Lab 1.4.4 predictions.
3. Write the flight-test report: configuration, test cards and results, plots from the log, measured against predicted performance, every anomaly with its cause, and the changes you would make.

DONE WHEN

- ☐ The mission flies on telemetry with packet loss under 2 % at the flying distances used.
- ☐ Measured hover power is within 15 % of your prediction, or the report explains the difference.
- ☐ The report is written and another engineer could repeat your tests from it.

EVIDENCE FOR THE NOTEBOOK

- Telemetry logs; the flight log; the report.

Problem set

1. Encode DShot600 throttle 1,046 without telemetry and verify your checksum. What is the highest frame rate with a 2 µs gap between frames, and how does that compare with your rate loop?
2. The receiver runs at 250 packets per second. What is the worst-case delay from a stick movement to a new channels frame at the UART, and to DShot output with your measured latencies?
3. From your thrust model, compute the hover throttle at 12.6 V and 10.5 V for the drone's measured mass, and the feedforward you would use.
4. A roll demand needs motor 1 at 110 % while collective is 70 %. Give two mixer strategies, compute the four outputs for each, and say which keeps the drone level.
5. Your altitude filter uses accelerometer noise 0.3 m/s^2 and radar noise 2 cm at 50 Hz. Estimate the steady-state uncertainty in height and the delay the filter adds.
6. The radar reads 3.00 m with roll 10° and pitch −8°. Compute the height over flat ground and the error from a 2° error in each angle.

7. The link fails at 3 m and your firmware disarms 0.5 s later. Estimate the impact speed and energy. At what height would you rather descend under control than disarm?
8. The 1.66 kHz fast path spends 35 µs reading the IMU by DMA, 60 µs in the estimator, 15 µs in the controllers, 5 µs in the mixer and 8 µs starting DShot. What is its utilisation, and what is left for the 500 Hz logger if it needs 120 µs per record?
9. The drone hovers at 135 W with an all-up mass of 1.25 kg. Using 80 % of the pack's 55 Wh, estimate the hover time. What reduces it in practice?

Deliverables and stretch

Deliverables. The architecture document; the flight firmware with its simulator and tests; the assembled, balanced and documented drone; the tuning logs; video and logs of the hover and altitude-hold flights; the flight-test report.

Stretch. Bidirectional DShot with rpm-tracking notch filters, and the vibration spectra before and after. A controlled descent on link loss using the altitude estimate. Position logging and position hold with the Matek GNSS module. A second telemetry ground station on the BeaglePlay. Replace one CloudPhoenix with your Lab 3.1.3 FOC firmware on a small custom ESC board.

COMPLETION CHECKLIST

- ☐ **Lab 3.5.1:** DShot to four ESCs over µDMA; 10,000 frames decoded; commands and ramp demonstrated.
- ☐ **Lab 3.5.2:** CRSF parser passing corruption tests; failsafe within 120 ms; latency measured.
- ☐ **Lab 3.5.3:** architecture written; every simulation test passing; fast path under 50 % of its period.
- ☐ **Lab 3.5.4:** assembled, balanced, wired and verified with propellers off; vibration checked.
- ☐ **Lab 3.5.5:** roll and pitch tuned on the rig within the step-response criteria.
- ☐ **Lab 3.5.6:** 60 s free hover within ±5°, on video and in the log, with anomalies explained.
- ☐ **Lab 3.5.7:** altitude hold within ±15 cm at three heights, surviving a radar dropout.
- ☐ **Lab 3.5.8:** telemetry mission flown; flight time against prediction; the report written.
- ☐ **Problem set:** all nine answered with units.
- ☐ **You can explain**, without notes: what each subsystem on the drone does and how it fails, why arming is a state machine, what DShot and CRSF put on the wire, how the altitude filter blends three sensors, and how you climbed the test ladder.

UNIT 3

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and labs repository, and you can do each review task below without notes. Date each line as you finish it. If someone who builds hardware for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
3.1	Motors, estimation and feedback control	☐	☐	
3.2	Radio links	☐	☐	
3.3	Microwave synthesis and FMCW radar	☐	☐	
3.4	Board design and bring-up	☐	☐	
3.5	The drone: integration, test and flight	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook.

- ☐ Explain the drone's control loops from gyroscope to propeller, with their rates, delays and phase margins, using your flight logs.
- ☐ Explain how you proved your receiver's sensitivity, what limited the measurement, and how you knew the leakage was small enough.
- ☐ Derive an FMCW radar's beat frequency, then choose a chirp for a 15 m altimeter with 5 cm resolution and check it against the IWRL6432's limits.
- ☐ Walk the bring-up of a new board from the microscope to running firmware, using one failure from your errata as the example.
- ☐ Present your flight-test report to someone who builds drones and answer their questions.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____